ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ · ВЕРСИЯ 1.0

Онлайн-деканат

Предварительная проверка студенческих документов и электронная очередь заявок в деканат. Распознавание на CPU за 4–6 секунд, без внешних API.

Karpov Stepan Team · UMIRHACK 2026
Карпов Степан — тимлид, full-stack · Михайлов Роман — ML-инженер
github.com/Stepa-Karpik/hack · dekanat.karpovprod.ru · октябрь 2026

Содержание

1. Обзор
2. Архитектура
3. Движки распознавания
4. Конвейер
5. Модель
6. Методы
7. Модели
8. Приложение
9. ИИ-ассистент
10. REST API
11. Оценка качества
12. Данные и обучение
13. Развёртывание
14. Безопасность и данные
15. Разработка

Обзор

Что это

Студент загружает справку или заявление — и через несколько секунд видит, всё ли в нём на месте. Сервис:

- читает документ и находит поля: ФИО, даты, номер, организацию и т. д.;
- находит подпись и печать, а если их нет — **подсвечивает место на странице, где они должны быть** («(подпись)», «М.П.»);
- проверяет комплектность по Положению и называет недостающие пункты;
- сверяет ФИО в документе с ФИО в профиле студента;
- отвечает на вопросы о документе в чате.

Дальше студент отправляет заявку в деканат. Сотрудник видит её во входящих, открывает, одобряет или возвращает с комментарием. Окончательное решение всегда принимает человек: сервис выполняет только предварительную проверку.

Встроенные типы документов (правила кейса — в `configs/participant_v1.json`):

Тип	Ключ	Обязательные сведения
Справка об обучении	`study_certificate`	ФИО, вуз, направление, форма обучения, начало и окончание обучения, номер, дата выдачи, подпись, печать
Заявление на академический отпуск	`academic_leave_application`	ФИО, группа, курс, причина, начало и окончание отпуска, дата заявления, подпись студента
Справка о практике	`internship_certificate`	ФИО, организация, начало и окончание практики, руководитель, оценка, номер, дата выдачи, подпись, печать

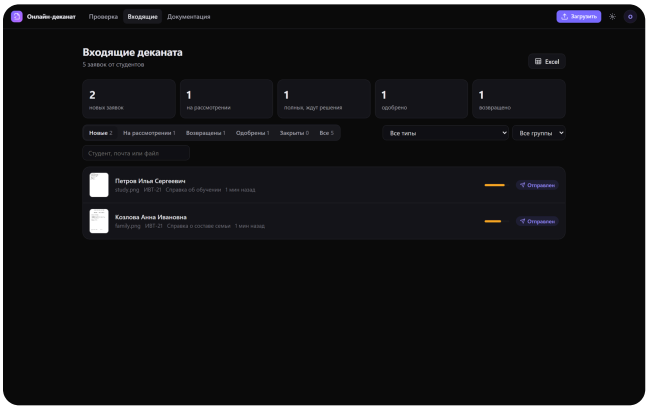
Новые типы администратор добавляет сам, без обучения модели, — в конструкторе.

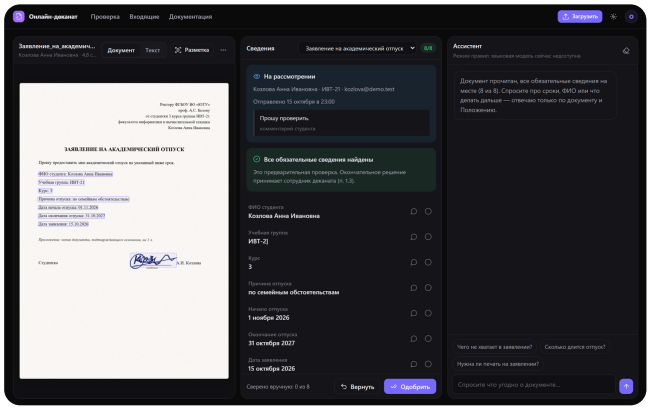
Форматы: PNG, JPG и PDF до 10 МБ.

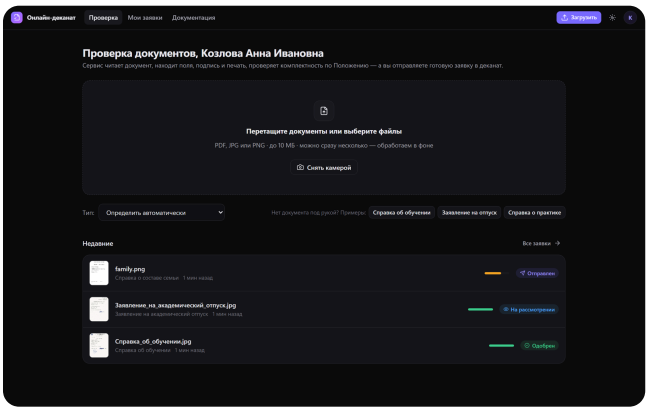
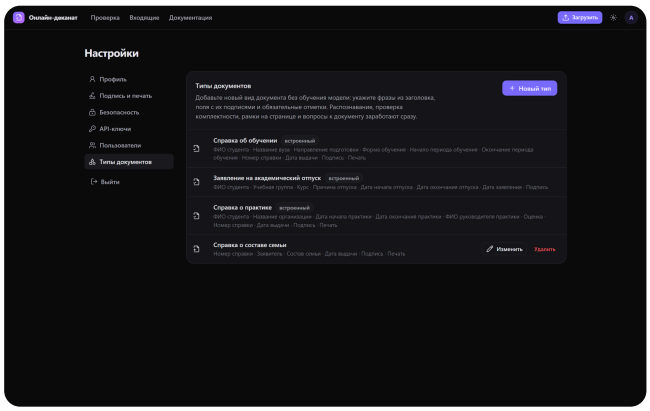
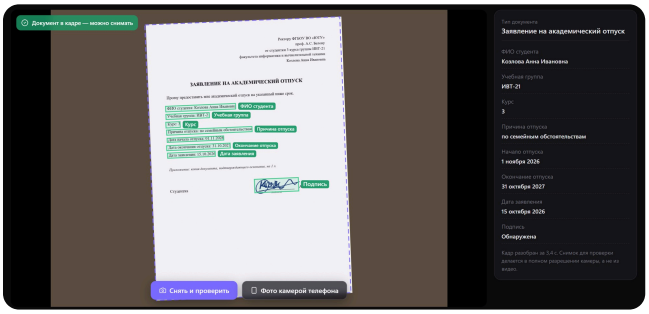
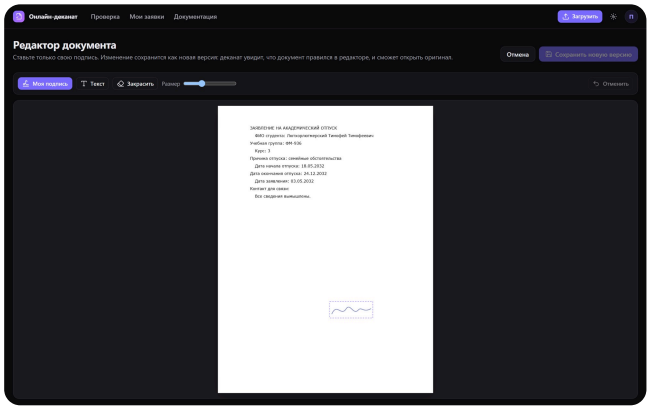
Возможности

Быстро на CPU	4,2 с на страницу на Ryzen 5 3600 (6 потоков, FP32, без квантования). Видеокарта не нужна
Заявки в деканат	Черновик → отправлен → на рассмотрении → одобрен / возвращён. Замена файла внутри той же заявки, полный журнал событий, закрытие после многих попыток

Честное предупреждение	Перед отправкой неполного документа: «Мы проверили документ: в нём не хватает подписи руководителя практики и печати организации — этого требуют пункты 5.2 и 5.3 Положения. С таким документом заявление, скорее всего, вернут…». Отправить всё равно можно
Подсветка недостающего	Красная пунктирная рамка там, где на бланке напечатано «(подпись)» или «М.П.», но отметки нет
Качество снимка	Если текст на фото читается плохо — снимок экрана, смаз, мелкий шрифт, — сервис предупреждает и просит переснять, а не теряет поля молча
Сверка ФИО	ФИО из документа сравнивается с профилем (с учётом падежей). Несовпадение — красная отметка и для студента, и для деканата
Живая камера	Наведите телефон на документ: тип и поля подсвечиваются прямо на видео (2–3,5 с на кадр), рамки следуют за дрожанием руки, подсказки «темно», «нерезко», «блик», «поднесите ближе». Снимок — в полном разрешении камеры
Редактор	Поставить свою подпись, добавить текст, закрасить область — с полупрозрачным «призраком» под курсором. Деканат ставит электронную печать. Каждая правка — новая версия, оригинал сохраняется, деканат видит пометку «изменён в редакторе»
Конструктор типов	Новый вид документа без обучения: фразы из заголовка, поля с подписями, обязательные отметки, пункт Положения
Входящие деканата	Счётчики, фильтры по статусу, типу и группе, поиск. Выгрузка в Excel (с тем, кто и когда принял решение) и PDF-протокол проверки
Роли и приглашения	Студент, сотрудник деканата, суперадмин. Сотрудники регистрируются только по одноразовой ссылке-приглашению
Ассистент	Локальная Qwen3.5-4B через Ollama. На CPU прямые вопросы («какой номер справки?», «чего не хватает?») отвечаются правилами мгновенно, модель — только для свободных вопросов
REST API и авто-одобрение	Ключи `dk_live_…`, загрузка, результаты, чат. Авто-одобрение полных документов — настройка профиля
Сайт на сервере, вычисления дома	Сервер без видеокарты может отдавать распознавание и чат на свой компьютер через исходящее WebSocket-соединение. Компьютер выключен — сервер распознаёт сам

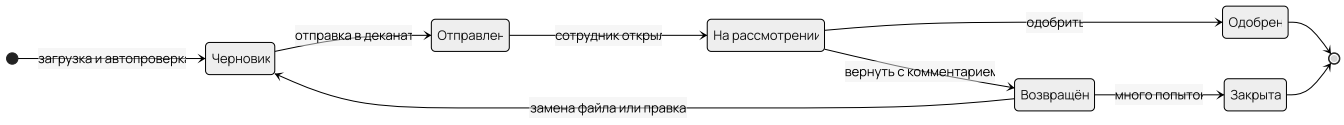




Входящие деканата: счётчики, фильтры по типу и группе, Excel	Заявка глазами сотрудника: комментарий студента, «Вернуть» / «Одобрить»
	
Загрузка: перетаскивание, несколько файлов, съёмка камерой	Конструктор: встроенные и добавленные типы
	
Живая камера: поля подсвечиваются прямо на видео	Редактор: своя подпись с превью под курсором

Роли и путь заявки

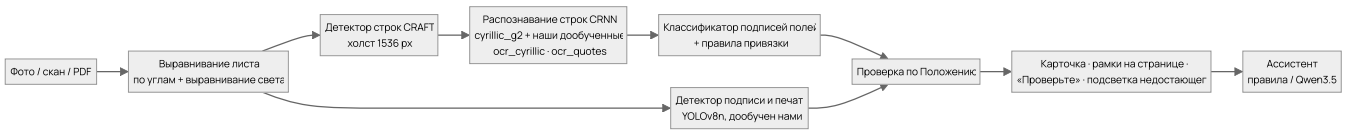
Роль	Как появляется	Что может
Студент	Регистрируется сам: ФИО «как в документах» и группа	Загружать, проверять, править свои документы, ставить **свою** подпись, отправлять в деканат
Сотрудник деканата	Только по одноразовой ссылке от суперадмина (действует 7 дней)	Входящие, решения, комментарии, электронная печать деканата, закрытие заявок, Excel и PDF
Суперадмин	Назначается на сервере	Всё перечисленное + приглашения, роли пользователей, конструктор типов



Каждый шаг пишется в журнал заявки: загружен, проверен автоматически, отправлен, взят на рассмотрение, возвращён, заменён файл, изменён в редакторе, одобрен, закрыт — кто и когда.

Как это работает

Быстрый CPU-движок (по умолчанию)



1. **Подготовка.** Лист находится по углам и выпрямляется по перспективе, неравномерное освещение выравнивается. Повороты на 90° и 180° вне условий кейса, поиск ориентации выключен ради скорости.
2. **Текст.** Детектор CRAFT ищет строки на холсте 1536 px: 1024 теряет даты, 2000 медленнее без выигрыша. Строки распознаются пакетами по 32. Цифровые поля перечитываются дообученной моделью `ocr_cyrillic.pt`, поля в «ёлочках» — моделью `ocr_quotes.pt`.
3. **Поля.** Классификатор узнаёт подписи полей («ФИО студента:», «Дата выдачи») по символьным n-граммам. Значения берутся справа от подписи или строкой ниже. Нормализация чинит типичные ошибки OCR: латинские буквы-двойники, «0» вместо «О» в префиксе номера, «]» вместо «1» в коде группы, кавычки.
4. **Подпись и печать.** YOLOv8n, дообученный на подписях и печатях. Если отметки нет, движок ищет напечатанное место для неё («(подпись)», «М.П.») и возвращает его рамку.
5. **Уверенность.** Значение с уверенностью распознавания ниже 0,5 помечается «Проверьте».
6. **Проверка.** Обязательность полей и пункты Положения берутся из конфигурации. Документ не «принимается» автоматически.

Живая камера

Браузер десять раз в секунду считает по уменьшенному кадру яркость, долю пересвеченных пикселей, резкость (дисперсия лапласиана), движение и сдвиг изображения относительно последнего распознанного кадра. Каждые 2–3 секунды самый резкий кадр за последнюю секунду (до 1920 px) уходит на `POST /api/live/scan`: тот же движок в быстром режиме (холст детектора 1024) возвращает тип, поля и рамки. Рамки переводятся из координат выпрямленного листа в координаты кадра через обратную матрицу перспективы, рисуются поверх видео и смещаются вместе с изображением. Снимок для проверки делается в полном разрешении сенсора (`ImageCapture.takePhoto`). Кадры живого режима нигде не сохраняются.

Конструктор типов документов

Администратор описывает тип в форме:

- название;
- фразы из заголовка («справка о составе семьи»). Сравнение идёт по основам слов, поэтому «состав семьи» находит и «о составе семьи»;
- поля: название, варианты подписи на бланке, вид значения (текст, ФИО, дата, номер), обязательность;
- нужна ли подпись и печать;

- пункт Положения.

Обучение не нужно: движок находит подписи полей в распознанном тексте и берёт значение рядом, отметки ищет тот же детектор. Однословная фраза («справка») не перехватывает документы, которые уже узнаны как встроенный тип. Проверка комплектности, рамки, предупреждения, Excel и чат для нового типа работают сразу.

PaddleOCR-VL (необязательный GPU-движок)

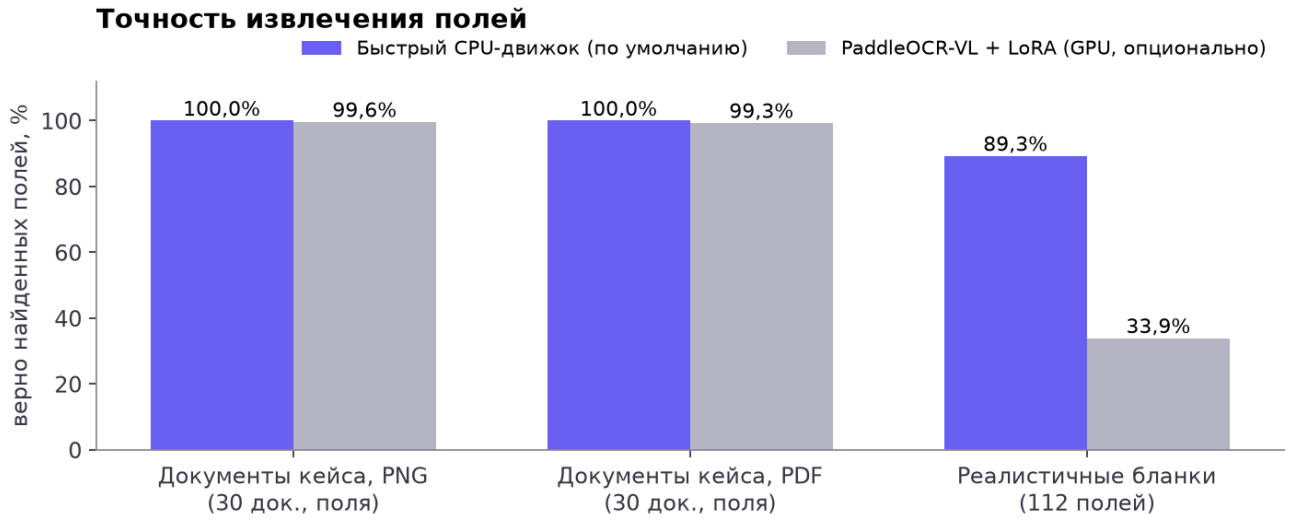
Исследовательская ветка проекта — одна визуально-языковая модель на всё: PaddleOCR-VL 1.6 (0,9 млрд параметров) с нашим LoRA-адаптером и головами подписи и печати. Собственный декодер со статическим KV-кэшем и CUDA graph даёт 3,5 с на странице на GPU. На CPU эта модель тратит около 2 минут на страницу, поэтому по умолчанию используется быстрый движок. Включается флагом `--engine gpu`. Подробно: docs/model.md.

Модели

Модель	Что делает	Происхождение	Размер
CRAFT (`craft_mlt_25k`)	Находит строки текста на странице	EasyOCR, используется как есть	79 МБ
CRNN `cyrillic_g2`	Распознаёт строку кириллицы	EasyOCR, используется как есть	15 МБ
`ocr_cyrillic.pt`	Перечитывает цифровые поля: даты, номера, курс	**дообучена нами** из `cyrillic_g2` на синтетике	15 МБ
`ocr_quotes.pt`	Перечитывает названия в кавычках: вуз, организация, оценка	**дообучена нами** из `ocr_cyrillic.pt`	15 МБ
`marks.pt`	Наш детектор подписи и печати (архитектура YOLOv8n от Ultralytics)	**дообучен нами** с весов COCO на подписях и процедурных печатях	6,5 МБ
`fields.joblib`	Узнаёт подписи полей в строках (TF-IDF по символам + логистическая регрессия)	**обучен нами** с нуля	0,6 МБ
`unified-experimental-natural-v5`	LoRA языковой части PaddleOCR-VL: чтение русских документов	**дообучен нами**, GPU-движок	2,5 МБ
`unified-experimental-mark-patches-v1`	Патч-голова подписи и печати с рамками на признаках PaddleOCR-VL	**обучена нами**, GPU-движок	1,3 МБ
PaddleOCR-VL 1.6	Базовая модель GPU-движка, скачивается отдельно	PaddlePaddle, используется как есть	1,9 ГБ
`qwen3.5:4b`	Ассистент в чате, через Ollama	Qwen, используется как есть (без дообучения)	3,3 ГБ

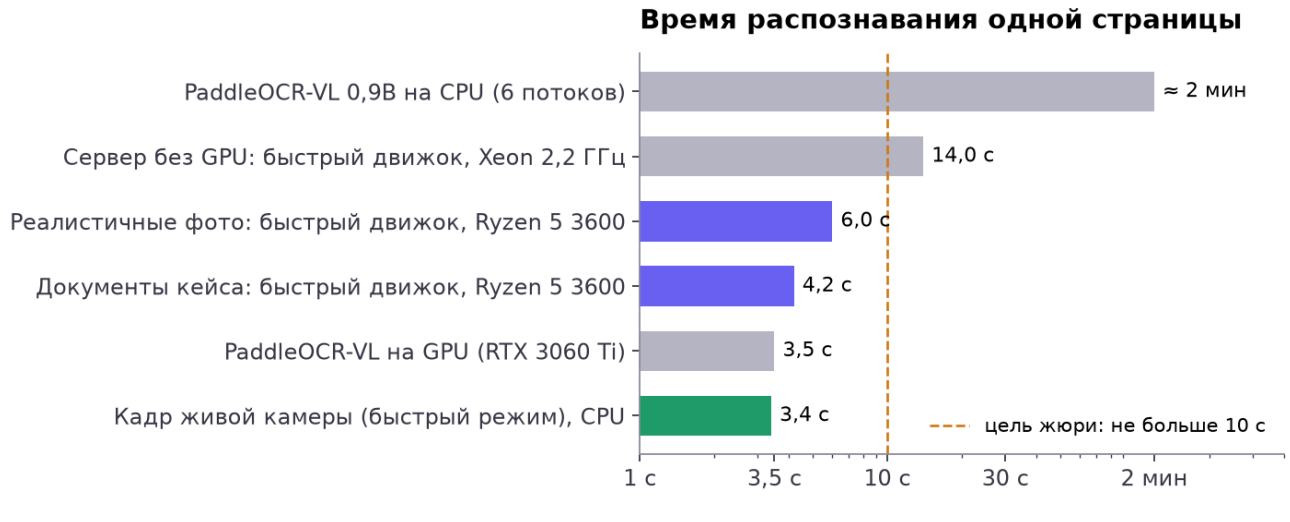
Каталог со всеми весами, включая экспериментальные и отклонённые адаптеры: models/README.md.

Качество и скорость

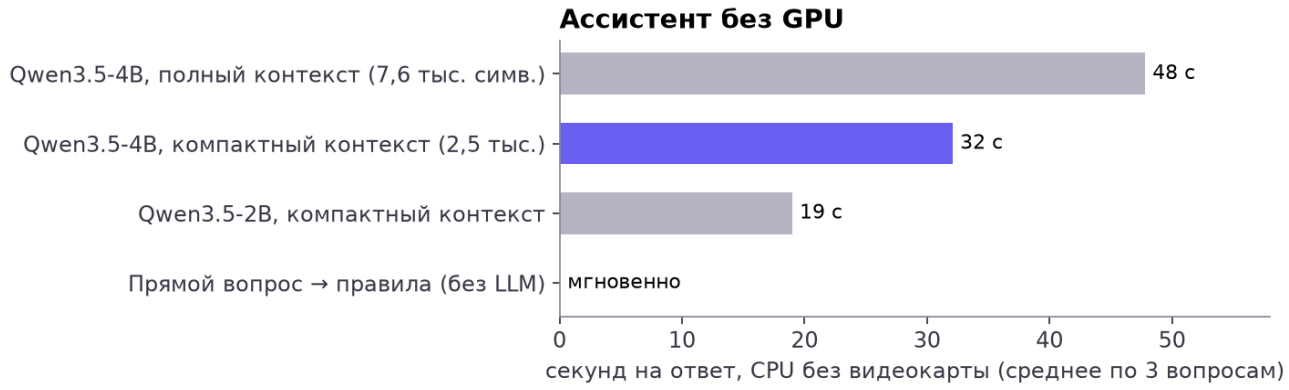


Набор	Быстрый CPU-движок	PaddleOCR-VL + LoRA (GPU)
Документы кейса, PNG: поля / комплектность / вопросы	**100% / 100% / 100%**	99,6% / 100% / 100%
Документы кейса, PDF: поля / комплектность / вопросы	**100% / 100% / 100%**	99,3% / 100% / 100%
Реалистичные бланки с фото, 112 полей	**100 верно, 1 ошибка**	38 верно

Документы кейса — синтетический набор организаторов: 30 документов, каждый в PNG и PDF. Он использовался **только для оценки**, в обучение не входил. Оценка — официальным `evaluate.py` из набора. Честная оговорка: несколько правил нормализации (кавычки, буквы-двойники) добавлены после разбора ошибок на этом наборе, поэтому 100% на нём — оценка сверху. Реалистичные бланки — отдельные фото вымышленных документов с рукописными подписями, тенями и перспективой.



Скорость	
Документ кейса, CPU Ryzen 5 3600	**4,2 с** в среднем (PNG), 4,4 с (PDF), максимум 4,7 с
Фото реалистичного бланка, тот же CPU	около 6 с
Кадр живой камеры	2–3,5 с
Сервер без GPU (Xeon 2,2 ГГц, 5 потоков)	около 14 с
PaddleOCR-VL на GPU RTX 3060 Ti	3,5 с



Без видеокарты даже небольшая языковая модель отвечает 20–50 секунд: почти всё время уходит на чтение длинного контекста. Поэтому на CPU ассистент работает в двух режимах. Прямые вопросы — значение поля, комплектность, кто принимает решение — сразу отвечает движок правил. Свободные вопросы идут в Qwen3.5-4B с компактным контекстом: поля плюс относящиеся к вопросу пункты Положения, без полного OCR-текста. Модель 2B быстрее, но чаще ошибалась, поэтому оставлена 4B. На GPU ответ занимает около 2 секунд.

Методика и все сравнения: docs/evaluation.md.

Быстрый старт

1. Окружение

```
git clone https://github.com/Stepa-Karpik/hack.git
cd hack
python -m venv .venv
```

На Windows окружение активируется командой `.venv\Scripts\activate`, на Linux и macOS — `source .venv/bin/activate`.

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements.txt -r requirements-app.txt
```

Для быстрого движка этого достаточно: веса EasyOCR и наши модели лежат в `models/`.

2. Ассистент (необязательно)

Нужна Ollama:

```
ollama pull qwen3.5:4b
```

Без Ollama чат отвечает движком правил.

3. Запуск

```
python -m scripts.app.server --port 8502 --engine cpu
```

Откройте http://127.0.0.1:8502. Данные (SQLite и файлы) сохраняются в `work/app-v1`. Первому администратору роль назначается на сервере (`UPDATE users SET role='admin' …`), дальше он приглашает сотрудников ссылками. На Windows то же самое делает `launch-app.ps1`.

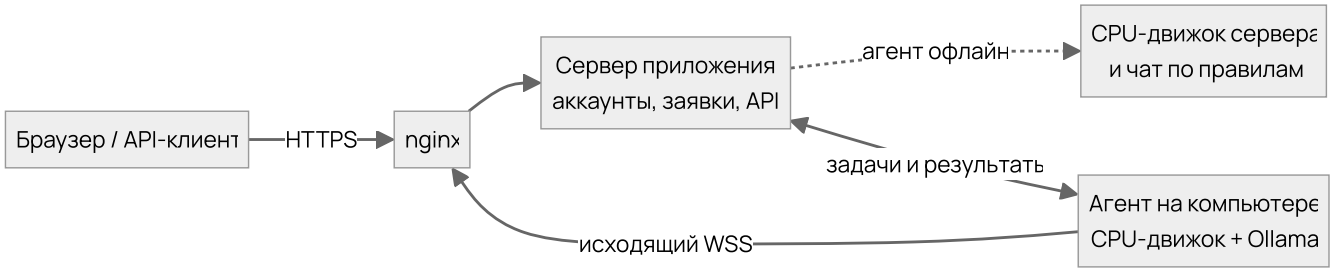
Для живой камеры с телефона сайт должен открываться по HTTPS: браузеры дают доступ к камере только на `https://` и `localhost`.

4. Результаты для проверки кейса

```
python -m scripts.app.make_results --kit <папка набора> --output results.json --engine cpu
--variant image
```

Обрабатываются все документы и вопросы набора. Результат — `results.json` в формате кейса.

5. Публичный сервер + вычисления дома



Сервер: `bash deploy/app-v2/install.sh app-v2.tar.gz`, по этапам `release venv model service nginx`. Компьютер: `launch-agent.ps1` или

```
python -m scripts.app.agent --server wss://<домен>/agent/ws --token-file work/agent-
v1/agent-token
```

Подробно: docs/deployment.md.

6. Тесты

```
python -m pytest -q
```

568 тестов. Модели в тестах заменены заглушками, поэтому веса и GPU не нужны.

Структура репозитория

```
├── scripts/
│   ├── app/              веб-приложение: сервер, заявки, роли, CPU-движок (fast_worker),
конструктор типов,
│   │                     Excel/PDF, живая камера, агент и брокер
│   ├── assistant/        ассистент: контекст (полный и компактный), схема ответа, проверка
привязки
│   ├── unified/          GPU-движок PaddleOCR-VL: чтение, граф-декодер, головы, обучение,
генераторы данных
│   ├── participantkit/   контракт кейса: формат результата, правила, адаптеры
│   └── *.py              обучение моделей CPU-движка (OCR, детектор, классификатор),
утилиты
├── dekanat/              библиотека CPU-движка: предобработка, OCR, извлечение, отметки,
правила
├── web-app/              интерфейс: ES-модули без сборки (проверка, входящие, редактор,
камера, конструктор)
├── configs/              Положение кейса и схемы документов
├── models/               собственные и дообученные модели
├── deploy/               развёртывание: systemd, nginx, установка, сборка релиза
├── tests/                568 тестов
└── docs/                 документация
```

Полная карта кода: docs/development.md.

Данные

Обучающие данные — **собственная синтетика** с вымышленными людьми и организациями:

- генераторы страниц: `dekanat/synthetic.py`, `scripts/unified/prepare_*.py`;
- фамилии из Wikidata (CC0);
- предложения из русской Википедии (CC BY-SA 4.0);
- реальные подписи из набора Ultralytics Signature (AGPL-3.0);
- процедурные печати.

Документы организаторов и реальные бенчмарки используются только для оценки. Наборы данных в репозиторий не входят: их можно воспроизвести генераторами. Подробно: docs/training.md.

Документация

Полная техническая документация одним PDF: docs/online-dekanat-documentation.pdf.

Раздел	Содержание
Приложение	Роли, заявки, редактор, камера, конструктор, входящие, выгрузки
Архитектура	Компоненты, путь документа, хранение, безопасность, протокол агента
Модель	GPU-движок: PaddleOCR-VL, адаптер, головы, граф-декодер
Движки распознавания	Конвейер и цельная модель: устройство, сравнение, выбор
Конвейер	Этапы, модули и параметры CPU-движка
Методы	Предобработка, каскад OCR, привязка полей, проверка, отметки, читаемость, камера
Модели	Карточки всех моделей и лицензии
Безопасность и данные	Роли, права, хранение, приватность
Данные и обучение	Генераторы, источники и лицензии, проверки утечек, рецепты
Оценка	Наборы, метрики, результаты
Ассистент	Устройство чата, проверка привязки, режимы CPU и GPU
REST API	Эндпоинты, авторизация, примеры
Развёртывание	Локально, сервер, агент, nginx, откат
Разработка	Окружения, тесты, карта кода
Журнал экспериментов	Сравнения вариантов цельной модели

Ограничения

- Рукописный текст полей не распознаётся (рукописная подпись — распознаётся как отметка).
- Многостраничные PDF не поддерживаются: нужна одна страница.
- Повороты на 90° и 180° не исправляются: снимайте документ ровно.
- Конструктор типов рассчитан на бланки вида «Подпись поля: значение». Свободный текст без подписей полей заполняется плохо.
- Обнаружение подписи или печати не проверяет их подлинность.
- Редактор позволяет ставить только свою подпись, а печать деканата — только сотрудникам. Каждая правка видна деканату, оригинал сохраняется.
- Проверка предварительная. Решение принимает сотрудник деканата.

Лицензии

Код и собственные модели распространяются под AGPL-3.0: детектор подписи и печати дообучен с помощью Ultralytics YOLOv8 (AGPL-3.0). Сторонние компоненты:

Компонент	Лицензия
EasyOCR (CRAFT, CRNN)	Apache-2.0
Ultralytics YOLOv8	AGPL-3.0
PaddleOCR-VL 1.6	Apache-2.0
Qwen3.5	Apache-2.0
DejaVu (шрифты PDF-протокола)	лицензия DejaVu (свободная, на основе Bitstream Vera)
Notyf	MIT
Lucide	ISC
Wikidata	CC0
Википедия	CC BY-SA 4.0

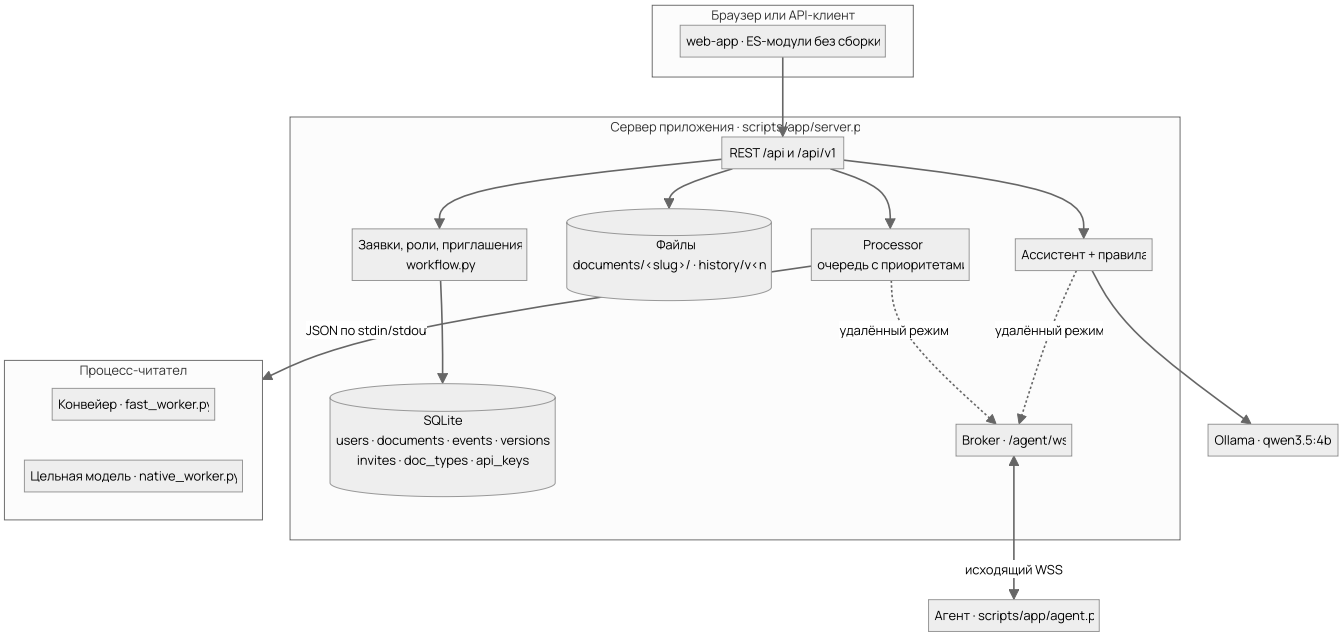
Все имена, организации и документы в примерах вымышлены.

Архитектура

Онлайн-деканат состоит из четырёх частей:

Часть	Назначение	Где работает
Веб-приложение	Аккаунты, роли, заявки, входящие деканата, API, интерфейс	Сервер (Starlette + SQLite)
Движок распознавания	Тип документа, поля, подпись и печать, рамки	Отдельный процесс-читатель на сервере или на агенте
Ассистент	Ответы на вопросы по распознанному документу	Ollama на сервере или на агенте; без неё — движок правил
Агент (необязательно)	Выполняет распознавание и чат на отдельном компьютере	Подключается к серверу исходящим WebSocket

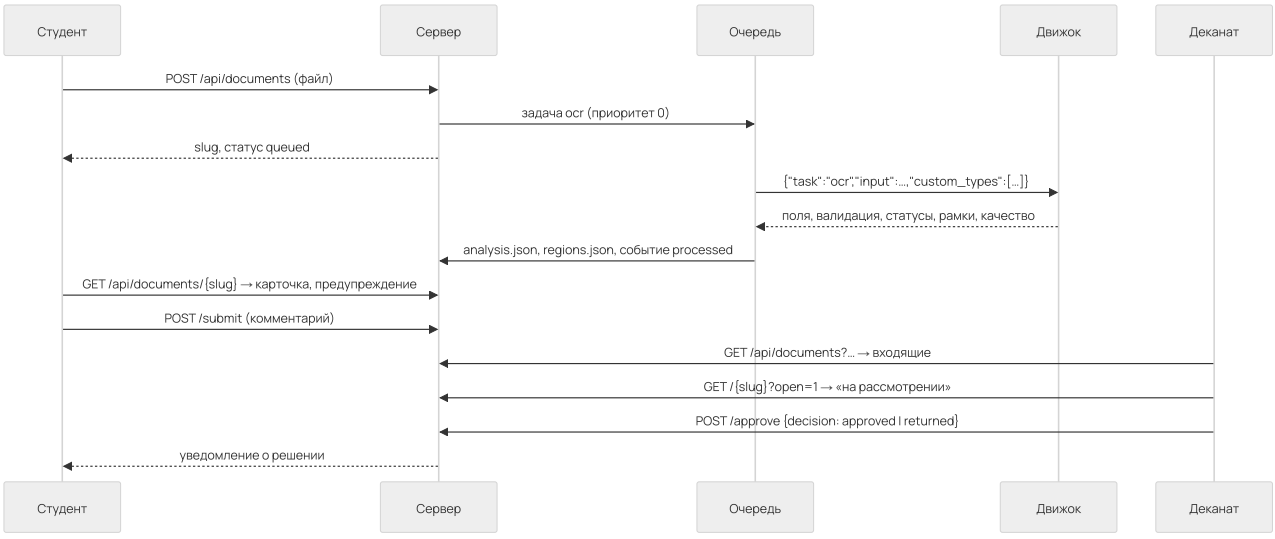
Компоненты



Модуль	Ответственность
`scripts/app/server.py`	Маршруты, авторизация и роли, загрузка, заявки, экспорт, живая камера, потоковый чат, раздача интерфейса
`scripts/app/store.py`	SQLite на стандартной библиотеке: пользователи, сессии, ключи, документы, события, версии, приглашения, типы
`scripts/app/workflow.py`	Состояния заявки и события, текст предупреждения, сверка ФИО, Excel, PDF-протокол

Модуль	Ответственность
`scripts/app/processing.py`	Очередь задач, процессы-читатели (`FastReader` , `Reader`), выбор устройства
`scripts/app/fast_worker.py`	Конвейер: распознавание, поля, рамки, места недостающих отметок, читаемость, быстрый режим для камеры
`scripts/app/custom_types.py`	Типы из конструктора: проверка формы, распознавание по заголовку, извлечение полей
`scripts/app/pipeline.py`	Приведение ответа движка к карточке документа: поля, статусы, «Проверьте», отметки
`scripts/app/remote.py`	Брокер агента, удалённый читатель с ожиданием переподключения и прогретым резервом, удалённый чат
`scripts/app/agent.py`	Агент: исходящее соединение, выполнение задач на локальном движке и Ollama
`scripts/assistant/core.py`	Контекст (полный и компактный), JSON-схема ответа, проверка привязки
`scripts/participantkit/`	Контракт кейса: формат результата, правила, адаптеры, `make_results`
`dekanat/`	Библиотека конвейера: предобработка, OCR, извлечение, отметки, проверка дат
`scripts/unified/`	Цельная модель: чтение, собственный декодер, головы, привязка полей, обучение

Путь документа



1. **Загрузка.** Файл сохраняется в `documents/<slug>/input.*` , создаётся заявка в состоянии «черновик» и событие `uploaded` .
2. **Распознавание.** Очередь передаёт задачу процессу-читателю. Вместе с задачей передаются конфигурации типов из конструктора. Читатель возвращает строку результата: тип, поля, проверку комплектности, статусы, уверенность, рамки и оценку читаемости. Выпрямленная страница сохраняется как `preview.png` : это геометрия всех рамок.

3. **Карточка.** `pipeline.analyze_fast` приводит результат к единому формату. Сохраняются `analysis.json` и `regions.json`, пишется событие `processed`.
4. **Отправка.** Студент отправляет заявку. Если чего-то не хватает, он видит предупреждение (`workflow.submit_warning`).
5. **Решение.** Сотрудник открывает заявку, и она переходит в состояние «на рассмотрении». Затем сотрудник одобряет или возвращает её с комментарием.
6. **Повторная попытка.** Замена файла или правка в редакторе архивирует прежнюю версию в `history/v<n>/` и запускает распознавание заново.

Состояния заявки



Хранение

Данные	Где
Пользователи, роли, сессии, ключи API	`app.sqlite3`: `users`, `sessions`, `api_keys`
Заявки	`documents`: состояние, попытки, версия, решение, комментарии
Журнал	`events`: тип события, автор, время, подробности
Версии файлов	`versions` + `documents/<slug>/history/v<n>/`
Приглашения	`invites`: одноразовый токен (хранится хэш), роль, срок, кем использовано
Типы из конструктора	`doc_types`: ключ, название, JSON-конфигурация
Подписи и печати профиля	`profiles/<user_id>/signature.png`, `seal.png`

Протокол агента

Агент подключается к `wss://<домен>/agent/ws` с токеном в заголовке `Authorization: Bearer`. Сообщения — JSON:

Направление	Сообщение
агент → сервер	`hello` с `info`: имя хоста, GPU, доступность ассистента, режим `compact`
сервер → агент	`task`: `ocr` / `chat` / `warm` с данными (файл в base64, тип, типы из конструктора, флаг `quick`)
агент → сервер	`chunk` (поток чата), `result` (`output` и `preview` в base64), `error`

Направление	Сообщение
обе стороны	`ping` / `pong`

Сервер устойчив к потере агента:

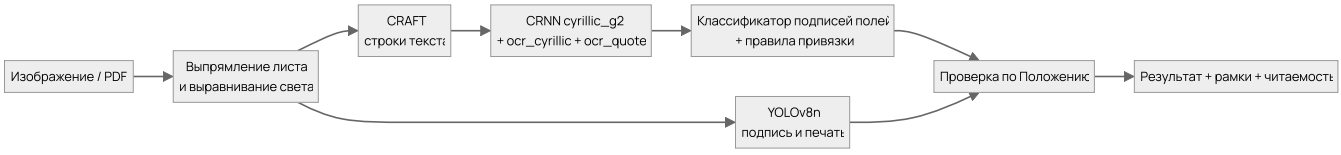
- **Короткий разрыв.** Если связь пропала меньше минуты назад или сервер только что запустился, задача до 20 секунд ждёт переподключения агента.
- **Агент выключен.** Задача выполняется резервным CPU-движком сервера. Резерв загружается при старте службы, поэтому холодного старта нет.

Движки распознавания

Онлайн-деканат поставляется с двумя движками распознавания. У них общий контракт результата: тип документа, поля, проверка комплектности, статусы, уверенность и рамки. Поэтому приложение, API, ассистент и выгрузки работают одинаково с любым из них.

	Конвейер	**Цельная модель**
Принцип	Специализированные модели и детерминированные правила	Одна визуально-языковая модель читает страницу целиком
Оборудование	CPU, видеокарта не нужна	GPU с 8 ГБ памяти (RTX 3060 Ti и выше)
Время на страницу	**4,2 с** (Ryzen 5 3600, 6 потоков), около 6 с на фото с телефона	**3,5 с** (RTX 3060 Ti)
Поля, набор кейса PNG / PDF	**100% / 100%**	99,6% / 99,3%
Поля, реалистичные фото бланков	**100 из 112**	38 из 112
Подпись и печать	YOLOv8n, дообученный	Патч-голова на признаках модели
Рамки полей	По строкам OCR, сразу с результатом	Задача `Spotting:` той же модели, в фоне
Типы из конструктора	Поддерживаются	Только встроенные типы
Живая камера	Быстрый режим, 2–3,5 с на кадр	—
Размер весов	≈ 140 МБ	≈ 1,9 ГБ + адаптер и головы
Флаг запуска	`--engine cpu` (по умолчанию)	`--engine gpu`

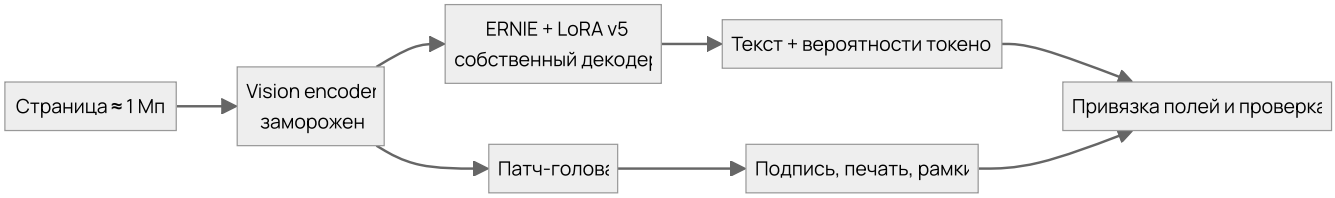
Конвейер (по умолчанию)



Конвейер раскладывает задачу на этапы. У каждого этапа своя небольшая модель или детерминированное правило. Поэтому он быстрый на CPU, предсказуемый и объяснимый: для каждого значения известна строка, из которой оно взято, и уверенность распознавателя. Новые типы документов добавляются без обучения, через конструктор.

Подробно: Конвейер, Методы.

Цельная модель



PaddleOCR-VL 1.6 (0,9 млрд параметров) читает страницу за один визуальный проход. Языковая часть дообучена адаптером LoRA на русских документах. Подпись и печать определяет патч-голова на тех же визуальных признаках. Собственный декодер со статическим KV-кэшем и CUDA graph выдаёт тот же текст, что и стандартная генерация, побайтно, но в 4–6 раз быстрее.

Подробно: Цельная модель.

Как выбрать

- **Нет видеокарты, или важна предсказуемая скорость** — конвейер. Это режим публичной установки и режим по умолчанию.
- **Есть GPU, документы с произвольной вёрсткой, нужен полный текст страницы для ассистента** — цельная модель.
- **Сервер без GPU и компьютер с GPU** — сервер принимает документы, агент на компьютере выполняет распознавание. См. Развёртывание.

Движок выбирается при запуске:

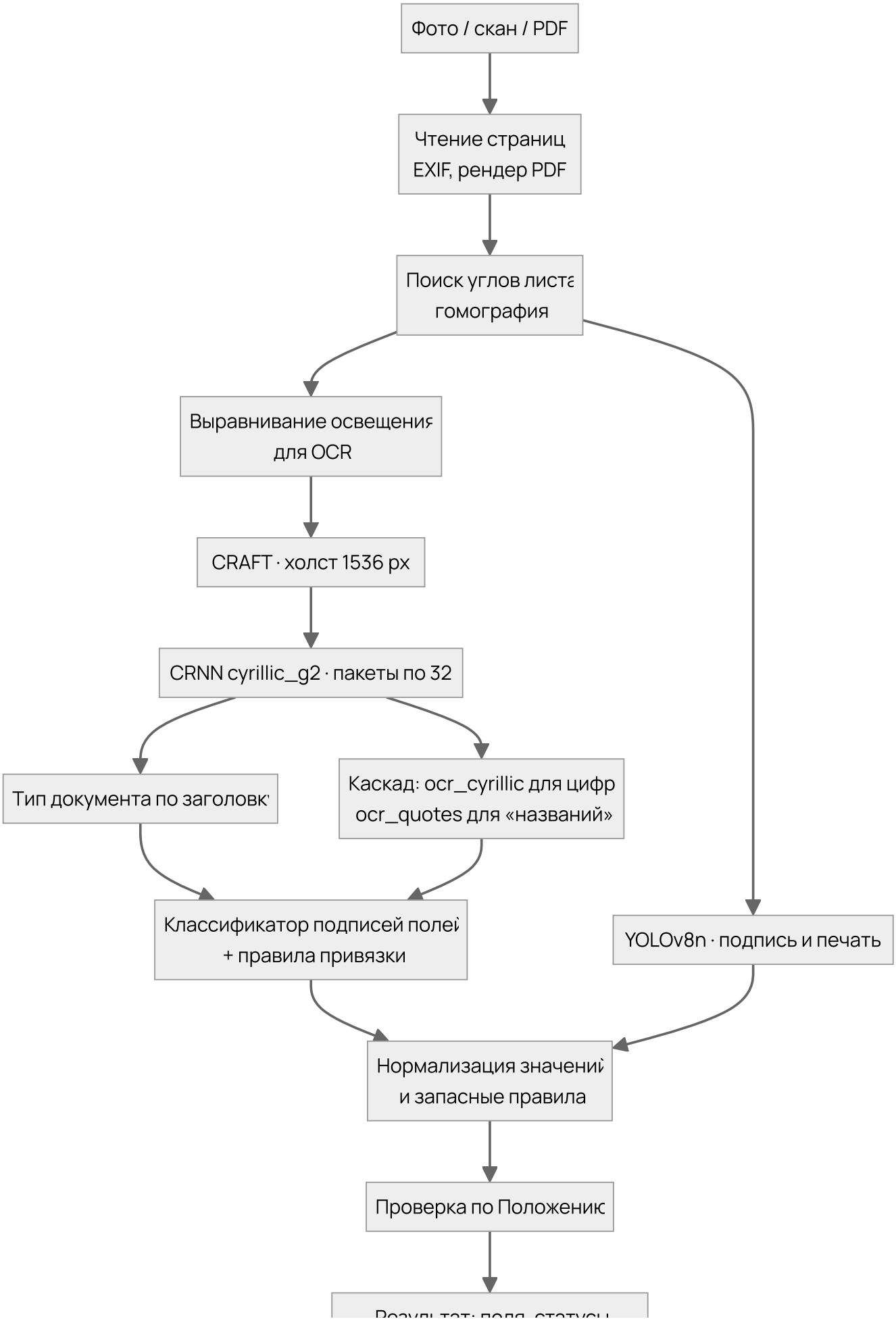
```
python -m scripts.app.server --engine cpu
```

```
python -m scripts.app.server --engine gpu
```

Для агента используется флаг `--engine cpu|gpu` в `scripts.app.agent`.

Конвейер

Конвейер — CPU-движок распознавания по умолчанию. Процесс-читатель `scripts/app/fast_worker.py` загружает все модели один раз, прогревается на образце и затем обрабатывает задачи по одной: один JSON на строку stdin, ответ — одна строка stdout.


Фото / скан / PDF
Чтение страниц
EXIF, рендер PDF
Поиск углов листа
гомография
Выравнивание освещения
для OCR
CRAFT · холст 1536 px
CRNN cyrillic_g2 · пакеты по 32
Тип документа по заголовк
Каскад: ocr_cyrillic для цифр
ocr_quotes для «названий»
Классификатор подписей полей
+ правила привязки
YOLOv8n · подпись и печать
Нормализация значений
и запасные правила
Проверка по Положению

Результат: поля, статусы, уверенность, рамки, читаемост

Этапы

Этап	Модуль	Устройство
Чтение файла	`dekanat/io.py`	PNG, JPG (с учётом EXIF-поворота) и PDF (рендер страниц)
Выпрямление	`dekanat/preprocessing.py`	Поиск четырёх углов яркого листа на контрастном фоне, гомография. Матрица `rectified_to_input` связывает выпрямленную страницу с исходным кадром
Освещение	`dekanat/preprocessing.py`	Деление на оценку фона. Применяется только для OCR, а не к сохраняемому изображению
Строки	EasyOCR CRAFT	Холст детектора 1536 px — выбран на наборе кейса: 1024 теряет даты, 2000 медленнее без выигрыша
Распознавание	EasyOCR CRNN `cyrillic_g2`	Строки читаются с полноразмерной страницы, пакетами по 32
Каскад цифр	`models/ocr_cyrillic.pt`	Перечитывает сомнительные цифровые поля: путаница 0/О, 3/З, коды групп, даты
Каскад кавычек	`models/ocr_quotes.pt`	Распознаватель с токенами «». Перечитывает название вуза, организации и оценку, если поле не найдено
Тип документа	`dekanat/extraction.py`	По заголовку и характерным фразам. Типы из конструктора — по фразам администратора
Поля	`dekanat/extraction.py`, `models/fields.joblib`	Классификатор узнаёт подписи полей, правила берут значение справа или строкой ниже
Проза	`dekanat/narrative.py`, `slots.py`	Значения из типовых предложений («Настоящая справка выдана …»)
Подпись и печать	`models/marks.pt`	YOLOv8n. Слабые кандидаты поверх печатной надписи «Подпись» отклоняются
Нормализация	`scripts/app/fast_worker.py`	Кавычки, латинские буквы-двойники, «0» в буквенном префиксе номера, «]» вместо «1» в коде группы
Запасные правила	`scripts/app/fast_worker.py`	Период «с … по …», прочитанный в другом порядке, причина отпуска. Заполняют только пустые поля
Проверка	`scripts/participantkit/format_v1.py`	Обязательные поля и пункты Положения из `configs/participant_v1.json`
Места отметок	`scripts/app/fast_worker.py`	Если подписи или печати нет, ищется напечатанное «(подпись)» или «М.П.», и возвращается рамка этого места

Этап	Модуль	Устройство
Читаемость	`scripts/app/fast_worker.py`	Средняя уверенность распознавателя и доля неуверенных строк

Производительность

Условия	Время на страницу
Ryzen 5 3600, 6 потоков, документы кейса PNG	4,2 с в среднем, максимум 4,6 с
То же, PDF	4,4 с в среднем
Фото реалистичного бланка	около 6 с
Кадр живой камеры, быстрый режим (холст 1024)	2–3,5 с
Сервер Xeon 2,2 ГГц, 5 потоков	около 14 с

Вычисления выполняются в FP32, без квантования. Детектор работает в формате памяти `channels_last`: на CPU это примерно на 20% быстрее при тех же результатах. Поиск ориентации на 90°/180° по умолчанию выключен (`DEKANAT_ORIENTATION=0`).

Параметры окружения

Переменная	По умолчанию	Назначение
`DEKANAT_DETECT_SIDE`	`1536`	Размер холста детектора строк
`DEKANAT_OCR_BATCH`	`32`	Размер пакета распознавателя
`DEKANAT_THREADS`	`6`	Потоки PyTorch
`DEKANAT_ORIENTATION`	`0`	Поиск поворота страницы
`DEKANAT_CPU_PYTHON`	`work/venv`	Интерпретатор окружения конвейера

Протокол процесса-читателя

Запрос:

```
{"task": "ocr", "input": "…/input.jpg", "output": "…/native-result.json",
 "document_type": "auto", "quick": false, "custom_types": []}
```

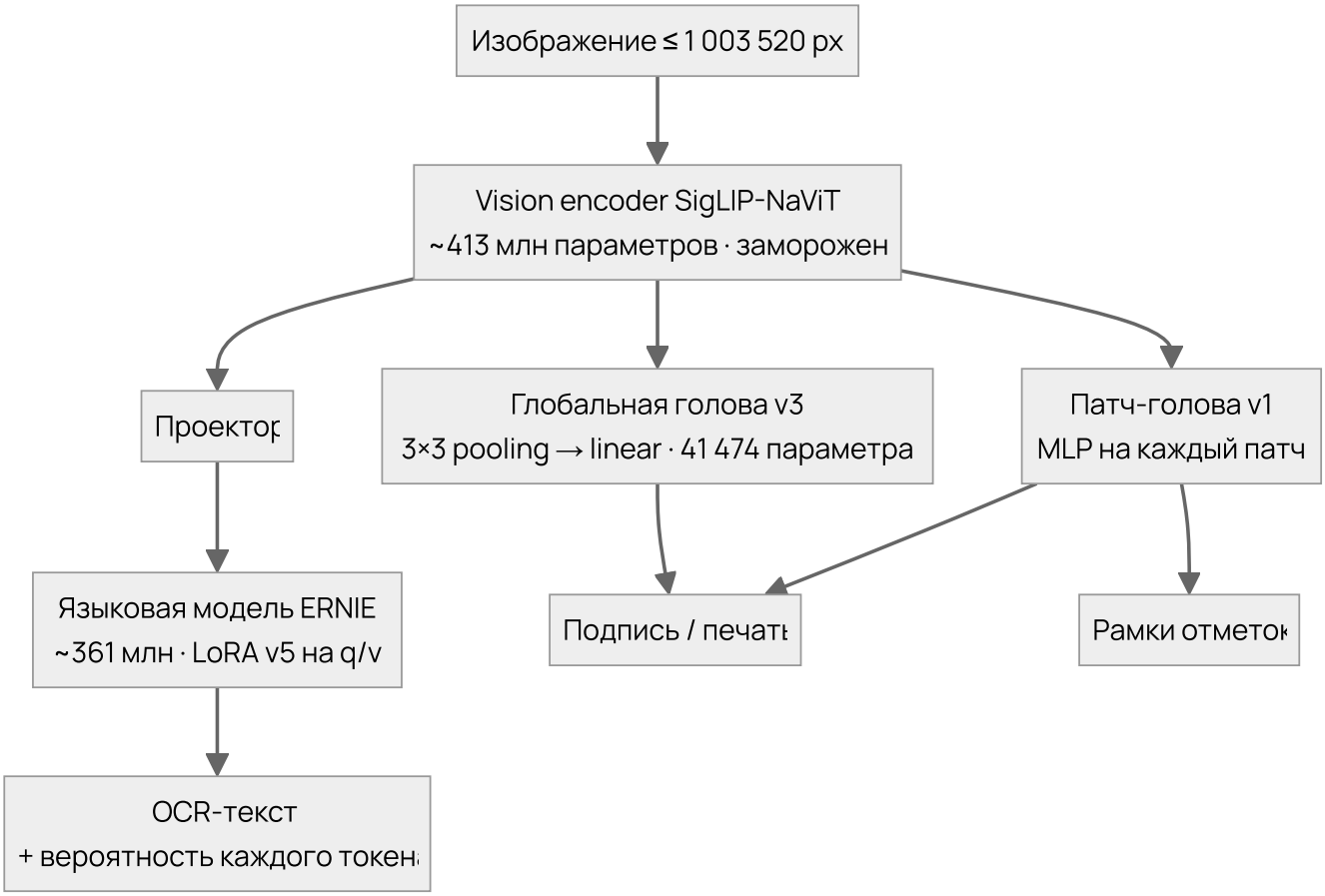
Ответ: `{"ok": true}`, результат записывается в `output`. Ключевые поля строки результата:

Поле	Содержание
`detected_type`	Официальный ключ типа или ключ типа из конструктора
`fields`, `validation`	Значения в формате кейса и проверка комплектности

Поле	Содержание
`statuses`, `confidence`	`found` / `not_found` / `conflict` и уверенность по каждому полю
`regions`	Нормализованные рамки полей; `kind: "missing"` — место недостающей отметки
`quality`	`mean_confidence`, `low_share`, `readable`, `warnings`
`live`	Только в быстром режиме: рамки в координатах кадра, контур листа, доля листа в кадре
`ocr_text`	Распознанный текст в порядке чтения

Модель

Весь документ читает **одна** нейросеть — PaddleOCR-VL 1.6. Подпись и печать определяют две небольшие головы на её же визуальных признаках. Второго OCR, детектора или отдельного энкодера нет. Один визуальный проход на страницу проверяется в коде: если энкодер вызван не ровно один раз, чтение завершается ошибкой.



Базовая модель

Модель	PaddlePaddle/PaddleOCR-VL-1.6, ревизия `c5630abae1d940eafe0697512a0325494b02ab42`
Параметры	905 601 648
Лицензия	Apache-2.0
Загрузка	Transformers 5.5.4, `trust_remote_code=False`. Удалённый код не выполняется, полный конвейер Paddle не нужен
Точность	Строгий FP32, TF32 выключен для matmul и cuDNN
Разрешение	`longest_edge = 1 003 520` пикселей (~1 Мп)
Задачи	`OCR:` — текст страницы, `Spotting:` — строки с координатами

Бюджет	1024 новых токена на страницу. Если он исчерпан, документ помечается, а пустые поля не заполняются догадками

Почему строгий FP32:

- результат воспроизводим между устройствами: CPU и GPU дают тот же текст, поля и отметки, расхождение оценок до $6 \cdot 10^{-5}$;
- BF16 не ускорял декодирование, потому что оно упирается в накладные расходы на каждый шаг, а не в арифметику.

Скачать веса: `python deploy/app-v2/fetch_base_model.py deploy/app-v2/base-model-manifest.json work/paddle-vl-base-v1`. Скрипт проверяет SHA-256 каждого файла.

Адаптер LoRA v5 (`models/unified-experimental-natural-v5`)

Дообучается только языковая часть. Vision encoder и графические головы заморожены. После обучения хэши всех 437 параметров vision совпадают, а признаки и оценки голов на контрольных изображениях совпадают бит в бит.

Параметр	Значение
Модули	`q_proj`, `v_proj` языковой модели
Ранг / α / dropout	8 / 16 / 0,05
Обучаемых параметров	626 688
Инициализация	Адаптер v4 (`unified-experimental-literal-values-fp32-v4`)
Данные	360 страниц v5 (естественный текст, 27 семейств шрифтов, разные подписи полей) и 120 страниц v4b для повторения
Шаги	240 (480 микробатчей, накопление 2), один проход
Оптимизатор	AdamW, LR $2 \cdot 10^{-5}$, разогрев 16 шагов, косинус до 0,1×, weight decay 0,01, clip 1,0, seed 42
Время	1680 с на RTX 3060 Ti, пик памяти 5,7 ГБ
SHA-256	`37ca4920…1317`

Как получены данные и что показала оценка: training.md, evaluation.md.

Глобальная графическая голова v3

Признаки всех патчей сводятся в сетку 3×3 (mean и max), всего 20 736 чисел. По ним одна линейная проекция даёт две оценки: «есть подпись» и «есть печать».

- 41 474 параметра;
- обучена в FP32 на 383 страницах;

- оценки не откалиброваны как вероятности.

Патч-голова v1 (`models/unified-experimental-mark-patches-v1`)

Глобальная голова не даёт координат. Она пропускала подписи поверх линии и иногда «видела» печать там, где её нет. Патч-голова оценивает каждый патч отдельно.

```
вход патча  = [признак патча (1152), среднее соседей 3×3 (1152)]   → 2304
MLP         = Linear(2304 → 128) → GELU → Linear(128 → 2)          → подпись, печать
карта       = sigmoid → сглаживание 3×3
наличие     = max(карта) ≥ 0,94
рамка       = связная область вокруг максимума, где p > max(0,4; 0,6·max)
```

Обучение	
Страницы	743 train: 360 новых `marks-v1`, `unified-data-v2` и `unified-real-marks-v1`
Сэмплинг	Все положительные патчи и до 700 отрицательных с каждой страницы
Потеря	BCE с `pos_weight` = доля отрицательных (не больше 30)
Оптимизатор	AdamW, LR $2 \cdot 10^{-3}$, weight decay 0,01, 3000 шагов по 4096 патчей, seed 42
Выбор порога	Только на val: перебор порога 0,20–0,94 и правила «решает патч-голова» или «любая из голов». Выбрано 0,94 и «решает патч-голова»

Набор `marks-v1` создан специально под ошибки старой головы. В нём есть:

- подпись поверх линии над «(подпись)»;
- круглые и прямоугольные печати;
- негативы: пустые строки подписи, «М.П.» без печати, цветные логотипы.

Генератор: `scripts/unified/prepare_mark_pages.py`.

Граф-декодер (`scripts/unified/fast_decode.py`)

Профилирование показало, что около 50 мс на токен уходило на Python и запуск ядер. Кроме того, FP32-ядро внимания плохо работает с одним запросом. Граф-декодер оставляет ту же жадную стратегию и ту же точность:

1. **Prefill** выполняется обычным образом в `StaticCache`.
2. **Шаг декодирования** записывается один раз как CUDA graph и затем воспроизводится.
3. **Внимание** групповое (GQA), прямо по кэшу, без копий `repeat_kv`. Оно зарегистрировано через `AttentionInterface` и `AttentionMaskInterface`.
4. **Вероятность выбранного токена** (максимум softmax) сохраняется для оценки уверенности.

При любой ошибке включается обычный `generate`.

Квалификация (`scripts/unified/qualify_fast_decode.py`)	Результат
OCR-текст на 156 страницах (60 организаторов, 96 v4b) против сохранённых выходов `generate`	**156/156 побайтно**
Поля и отметки на документах организаторов	60/60 совпали, разница оценок 0,0
Время на страницу	3,5 с в среднем (2,7–4,6 с) против 15–22 с; из них vision около 1,7 с
Декодирование	57 → 14,5 мс на токен

Привязка полей

Значения ищутся в прочитанном тексте детерминированно (`scripts/unified/text_binding.py`):

- **Подписи полей.** Ищутся явные подписи схемы: «ФИО студента:», «Номер справки:», «Дата начала практики:» и их варианты. Значение — текст после подписи до конца логической строки.
- **Layout v1.** Если модель склеила две соседние пустые метки, они разделяются (`separate_empty_labels`).
- **Layout v2.** Перед следующей полной меткой схемы пробел заменяется переводом строки (`separate_label_runs`). Длина текста и все непробельные символы при этом сохраняются.
- **Проза.** Сведения справки об обучении, записанные одним предложением, восстанавливаются ограниченными шаблонами (`prose_binding.py`). Заполняются только пустые поля.
- **Даты.** Приводятся к ISO, если записаны полностью. Неполная дата не достраивается.
- **Конфликт.** Если одна метка встречается с разными значениями, поле получает статус `conflict` и пометку «Проверьте».

Тип документа определяется по заголовку в прочитанном тексте. Если выбранный вручную тип не совпадает с заголовком, текстовые поля остаются пустыми. Подставлять значения из «чужой» схемы нельзя.

Уверенность и «Проверьте»

Для каждого найденного значения берётся минимум вероятностей его токенов. Токены подписи поля не учитываются.

Поле	Порог
`student_name` , `supervisor_name`	0,8
остальные текстовые	0,4

Порог выбран на v4b val/test, где известны правильные значения, и проверен на документах организаторов (`scripts/app/calibrate_review.py`):

	Найдено ошибочных значений	Лишних пометок на верных
v4b val/test	8 из 10	63 из 570 (11%)
Документы организаторов	3 из 3	48 из 407 (12%)

«Проверьте вручную» у подписи или печати появляется, если в тексте есть место для отметки («(подпись)», «М.П.»), а отметка не найдена.

Разметка полей

Задача `Spotting:` той же модели возвращает строки с четырёхугольниками. `field_regions.py` сопоставляет значения полей со строками и строит рамки. Рамки подписи и печати даёт патч-голова. Разметка выполняется в фоне после распознавания и уступает очередь новым загрузкам.

Методы

Раздел описывает методы, на которых построена проверка документа: от подготовки изображения до проверки по Положению и живой камеры. Если не сказано иное, речь идёт о конвейере. Особенности цельной модели описаны в model.md.

1. Подготовка изображения

Выпрямление листа. Страница ищется как самый крупный яркий четырёхугольник на более тёмном фоне. По четырём углам строится гомография, и лист приводится к прямоугольнику. Обратная матрица `rectified_to_input` сохраняется: по ней рамки с выпрямленной страницы переносятся обратно на исходное фото или кадр камеры.

Освещение. Фон оценивается сильным размытием и делится на него. Тени и неравномерный свет выравниваются. Коррекция применяется только к копии для распознавания: пользователю показывается исходная страница.

Оценка качества. Для страницы считаются разрешение, дисперсия лапласиана (резкость) и соотношение градиентов по осям (направленный смаз). Предупреждения передаются в результат.

2. Распознавание текста

Детектор строк CRAFT работает на холсте 1536 px. Распознаватель читает вырезки строк с полноразмерной страницы: уменьшается только холст детектора, но не сам текст.

Каскад распознавателей. Базовый распознаватель `cyrillic_g2` читает все строки. Два дообученных распознавателя применяются точечно:

Распознаватель	Когда применяется	Что исправляет
`ocr_cyrillic.pt`	Сомнительные цифровые поля: даты, номера, курс, коды групп	Путаница 0/O, 3/З, 1/I, пропуски разделителей
`ocr_quotes.pt`	Название вуза, организации и оценка не найдены	Названия в «ёлочках»: у базового распознавателя нет этих символов

Каскад включается только для полей, где он улучшает результат, — это зафиксировано в `models/ocr_selection.json` и `quote_selection.json` вместе с SHA-256 весов.

3. Тип документа

Тип определяется по заголовку и характерным фразам распознанного текста: «справка об обучении», «академический отпуск», «справка о прохождении практики». Для типов из конструктора используются фразы администратора, сравнение идёт по основам слов в верхней половине

страницы. Если документ уже узнан как встроенный тип, пользовательский тип может перехватить его только фразой из двух и более слов.

Тип можно выбрать вручную. Тогда поля другого типа не подставляются: если заголовок не совпадает, поля остаются пустыми, а пользователь видит подсказку.

4. Привязка полей

Значение поля — это текст рядом с его подписью на бланке.

1. **Подписи полей.** Классификатор `fields.joblib` (TF-IDF по символьным 2–5-граммам и логистическая регрессия) определяет, какое поле подписано в строке. Он устойчив к ошибкам OCR: обучен на префиксах с имитацией типичных искажений.
2. **Значение** берётся справа от подписи до конца строки или строкой ниже, если справа пусто.
3. **Проза.** Типовые предложения («Настоящая справка выдана … в том, что … является студентом …») разбираются ограниченными шаблонами.
4. **Запасные правила.** Срабатывают, только если поле ещё пустое:
   - период «с … по …» — первые две возрастающие даты после слова «отпуск», «практика» или «обучение»;
   - причина отпуска — по типовой формулировке.
5. **Конфликт.** Если одна подпись встречается с разными значениями, поле получает статус `conflict` и пометку «Проверьте».

Значения не дополняются и не угадываются: неполная дата не достраивается, отсутствующее поле остаётся «не найдено».

5. Нормализация

Правило	Пример
Латинские буквы-двойники внутри русских слов	«с уходом» → «с уходом»
Ноль в буквенном префиксе номера	«0Б-1003» → «ОБ-1003»
Символ вместо единицы в коде группы	«ИВТ-2]» → «ИВТ-21»
Кавычки, прочитанные как буквы	`кПарус>` → «Парус»
Подпись поля в значении руководителя	«от организации: Иванов И. И.» → «Иванов И. И.»
Даты	«05.03.2026», «5 марта 2026 года» → `2026-03-05`

6. Проверка по Положению

Обязательные поля каждого типа и пункты Положения заданы в `configs/participant_v1.json`. Проверка возвращает:

- статус `document_ok` или `missing_fields`;

- список недостающих полей;
- источники — пункты Положения.

Для студента недостающие поля превращаются в обычный текст с родительным падежом и номерами пунктов (`workflow.submit_warning`).

7. Уверенность и «Проверьте»

Уверенность значения — минимальная уверенность распознавателя по строкам, из которых оно собрано. Ниже 0,5 значение помечается «Проверьте». Пометка показывает, где нужен взгляд человека, но значение не скрывает.

8. Подпись и печать

Детектор YOLOv8n ищет на выпрямленной странице два класса: подпись и печать. Кандидат с низкой оценкой, который перекрывает напечатанное слово «Подпись», отклоняется. Для типов из конструктора отклоняется кандидат, лежащий на уверенно распознанной строке текста: так жирный заголовок не принимается за подпись.

Место недостающей отметки. Если отметки нет, в тексте ищется напечатанное место для неё: «(подпись)», «подпись студента / руководителя», «М.П.», «место печати». Рамка этого места возвращается с `kind: "missing"` и подсвечивается красным на странице и в живой камере.

9. Читаемость снимка

Для строк длиной от 4 символов считаются средняя уверенность распознавателя и доля строк с уверенностью ниже 0,4. Снимок считается читаемым, если средняя уверенность не ниже 0,5 и неуверенных строк не больше 40%. Пороги выбраны на снимках и сканах: у пригодных снимков 0,68–0,88 и не больше 10% неуверенных строк, у снимка экрана монитора 0,31 и 74%. Нечитаемый снимок получает предупреждение на странице документа и в тексте перед отправкой.

10. Сверка ФИО

ФИО из документа сравнивается с ФИО в профиле студента. Сравниваются фамилия и имя по основам из трёх первых букв, в любом порядке. Поэтому «Козловой Анны» совпадает с «Козлова Анна». При несовпадении студент и сотрудник видят красную отметку.

11. Типы из конструктора

Конфигурация типа хранит:

- название;
- фразы заголовка;

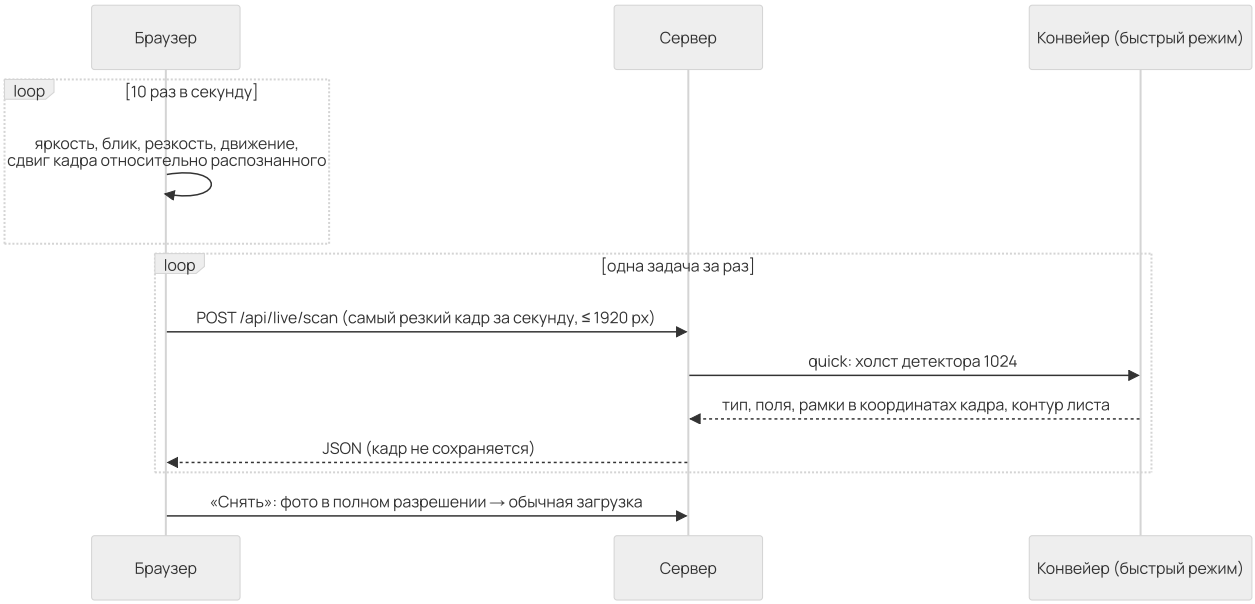
- поля: подпись и её варианты, вид значения (`text` , `name` , `date` , `number`), обязательность;
- требования к подписи и печати;
- пункт Положения.

Движок находит подпись поля в строке с допуском к регистру, «ё/е» и латинским двойникам. Значение приводится к виду поля:

- дата — ISO;
- ФИО — 2–3 слова с заглавной буквы или фамилия с инициалами;
- номер — первая группа символов с цифрой.

Конфигурации передаются движку с каждой задачей, поэтому одинаково работают локально и через агента.

12. Живая камера



- **Качество в браузере.** По уменьшенному кадру считаются:
  - средняя яркость;
  - доля пересвеченных пикселей на неравномерно белом листе (блик);
  - дисперсия лапласиана (резкость);
  - сглаженная разность соседних кадров (движение).
- **Дрожание рук.** Сдвиг текущего кадра относительно распознанного оценивается сопоставлением блоков в окне ±8 пикселей миниатюры. Рамки смещаются вместе с изображением. Если сцена сменилась, рамки скрываются.
- **Снимок.** `ImageCapture.takePhoto()` даёт фото с сенсора в полном разрешении. Если API недоступен, используется кадр видеопотока в максимальном разрешении. Кнопка «Фото камерой телефона» открывает системное приложение камеры.
- **Подсказки сервера.** Ответ содержит долю листа в кадре и признак читаемости. По ним показываются «поднесите ближе» и «текст читается плохо».

Модели

В системе девять моделей. Шесть из них — собственные: четыре дообучены на наших данных, две обучены с нуля. Обучающие данные — собственная синтетика с вымышленными людьми и организациями (training.md). Документы организаторов в обучение не входили и используются только для оценки.

Модель	Движок	Назначение	Происхождение	Размер
CRAFT `craft_mlt_25k`	Конвейер	Поиск строк текста	EasyOCR, используется как есть	79 МБ
CRNN `cyrillic_g2`	Конвейер	Распознавание строки кириллицы	EasyOCR, используется как есть	15 МБ
`ocr_cyrillic.pt`	Конвейер	Перечитывание цифровых полей	**Дообучена** из `cyrillic_g2`	15 МБ
`ocr_quotes.pt`	Конвейер	Перечитывание названий в кавычках	**Дообучена** из `ocr_cyrillic.pt`	15 МБ
`marks.pt`	Конвейер	Подпись и печать	**Дообучена**: архитектура YOLOv8n от Ultralytics, старт с весов COCO	6,5 МБ
`fields.joblib`	Конвейер	Классификатор подписей полей	**Обучена с нуля**	0,6 МБ
PaddleOCR-VL 1.6 + `unified-experimental-natural-v5`	Цельная модель	Чтение страницы	Базовая модель PaddlePaddle; адаптер LoRA **дообучен**	1,9 ГБ + 2,5 МБ
`unified-experimental-mark-patches-v1`	Цельная модель	Подпись и печать с рамками	**Обучена с нуля** на признаках модели	1,2 МБ
Qwen3.5-4B (`qwen3.5:4b`)	Ассистент	Ответы на вопросы	Qwen, используется как есть через Ollama	3,3 ГБ

Карточки моделей

`ocr_cyrillic.pt` — распознаватель цифровых полей

Задача	Распознавание строк с датами, номерами, курсом и кодами групп
Основа	EasyOCR `cyrillic_g2` (VGG + BiLSTM + CTC)
Обучение	4000 синтетических фрагментов строк с имитацией съёмки. Свёрточная часть заморожена, обучаются рекуррентные слои и выходной слой
Использование	Каскад: перечитывает только сомнительные цифровые поля

Отбор	Принят по аудиту разработки: поля 97,2% → 99,2%, полностью верные карточки 83,3% → 94,4% (`models/ocr_selection.json`)
Код	`scripts/train_ocr.py` , `scripts/train_official_ocr.py`

`ocr_quotes.pt` — распознаватель названий в кавычках

Задача	Названия вуза и организации, оценка в «ёлочках»
Основа	`ocr_cyrillic.pt` с двумя добавленными токенами `«` и `»`
Обучение	Синтетические строки с кавычками. LR $2 \cdot 10^{-6}$ для свёрточной части, 10^{-5} для BiLSTM, 10^{-4} для выходного слоя
Использование	Только если поле не найдено и есть явный контекст: справка об обучении или практике
Отбор	57 → 61 верных значений на отобранных случаях разработки (`models/quote_selection.json`)
Код	`scripts/train_quote_ocr.py`

`marks.pt` — детектор подписи и печати

Задача	Детекция двух классов: подпись, печать
Архитектура	YOLOv8n (Ultralytics, AGPL-3.0)
Обучение	Старт с весов COCO. 383 страницы: реальные подписи из набора Ultralytics Signature и процедурные печати разных форм, 640 px
Защита от ложных срабатываний	Слабый кандидат поверх напечатанного «Подпись» отклоняется. Для типов из конструктора отклоняется кандидат на уверенно распознанной строке
Код	`scripts/train_detection.py` , `scripts/participantkit/mark_guard.py`

`fields.joblib` — классификатор подписей полей

Задача	Определить, какое поле подписано в начале строки («ФИО студента:», «Дата выдачи»)
Модель	TF-IDF по символьным 2–5-граммам (до 15 000 признаков) + логистическая регрессия (C = 12, сбалансированные классы)
Обучение	21 802 синтетических префикса с имитацией ошибок OCR
Код	`scripts/train_fields.py`

Адаптер LoRA `unified-experimental-natural-v5`

Основа	PaddleOCR-VL 1.6, 905 млн параметров, ревизия `c5630ab…`
Модули	`q_proj` , `v_proj` языковой модели; ранг 8, α 16, dropout 0,05

Обучаемых параметров	626 688
Данные	360 страниц с естественным текстом, 27 семейств шрифтов, разные подписи полей, плюс 120 страниц повторения
Оптимизация	AdamW, LR $2 \cdot 10^{-5}$, 240 шагов, косинусный спад, seed 42
Время обучения	28 минут на RTX 3060 Ti
Подробно	model.md

Патч-голова `unified-experimental-mark-patches-v1`

Вход	Признак каждого патча vision encoder и среднее его соседей 3×3 (2304 числа)
Модель	MLP 2304 → 128 → 2, сглаживание карты 3×3
Решение	Отметка есть, если максимум карты ≥ 0,94. Рамка — связная область вокруг максимума
Обучение	743 страницы: подписи поверх линий, круглые и прямоугольные печати, негативы с «М.П.» без печати. AdamW, 3000 шагов
Подробно	model.md

Qwen3.5-4B — ассистент

Используется без дообучения через Ollama. Отвечает в JSON по схеме: вид ответа, текст, поля-источники, пункты Положения. Каждый ответ проверяется на привязку к документу. Подробно: assistant.md.

Лицензии

Компонент	Лицензия
EasyOCR (CRAFT, CRNN)	Apache-2.0
Ultralytics YOLOv8	AGPL-3.0 — поэтому проект распространяется под AGPL-3.0
PaddleOCR-VL 1.6	Apache-2.0
Qwen3.5	Apache-2.0
Собственные модели и адаптеры	AGPL-3.0

Приложение

Одностраничное приложение `web-app/` : ES-модули без сборки. Подключены Notyf 3.10 (MIT) для уведомлений и Lucide 0.469 (ISC) для иконок. Есть тёмная и светлая темы.

Роли

Роль	Как появляется	Что видит и может
Студент	Регистрируется сам. Указывает ФИО «как в документах» и учебную группу	Свои документы и заявки, проверка, редактор (только своя подпись), отправка в деканат
Сотрудник деканата	Регистрация только по ссылке-приглашению	«Входящие» со всеми отправленными заявками, решения, электронная печать деканата, закрытие заявок, Excel, PDF-протокол
Суперадмин	Назначается на сервере	Всё перечисленное, приглашения, роли пользователей, конструктор типов документов

Черновики студентов сотрудникам не видны: заявка попадает во входящие только после «Отправить в деканат».

Приглашения. Суперадмин создаёт ссылку `/register?invite=…` в разделе «Настройки → Пользователи». Ссылка одноразовая и действует 7 дней. Её можно отозвать. Аккаунт, созданный по ней, сразу получает роль сотрудника деканата.

Адреса

Адрес	Страница
`/`	Загрузка: перетаскивание нескольких файлов, тип «авто» или вручную, «Снять камерой», примеры, недавние заявки
`/check/<id>`	Заявка: документ, поля, подсветка, предупреждения, решение деканата, чат
`/check/<id>/edit`	Редактор документа
`/camera`	Живая камера
`/history`	Студенту — «Мои заявки», сотруднику — «Входящие деканата»
`/profile`	ФИО, группа, авто-подтверждение
`/profile/marks`	Своя подпись (нарисовать или загрузить), печать деканата для сотрудников
`/profile/users`	Приглашения и роли (суперадмин)
`/profile/types`	Конструктор типов документов (суперадмин)
`/profile/security` , `/profile/api`	Пароль, API-ключи

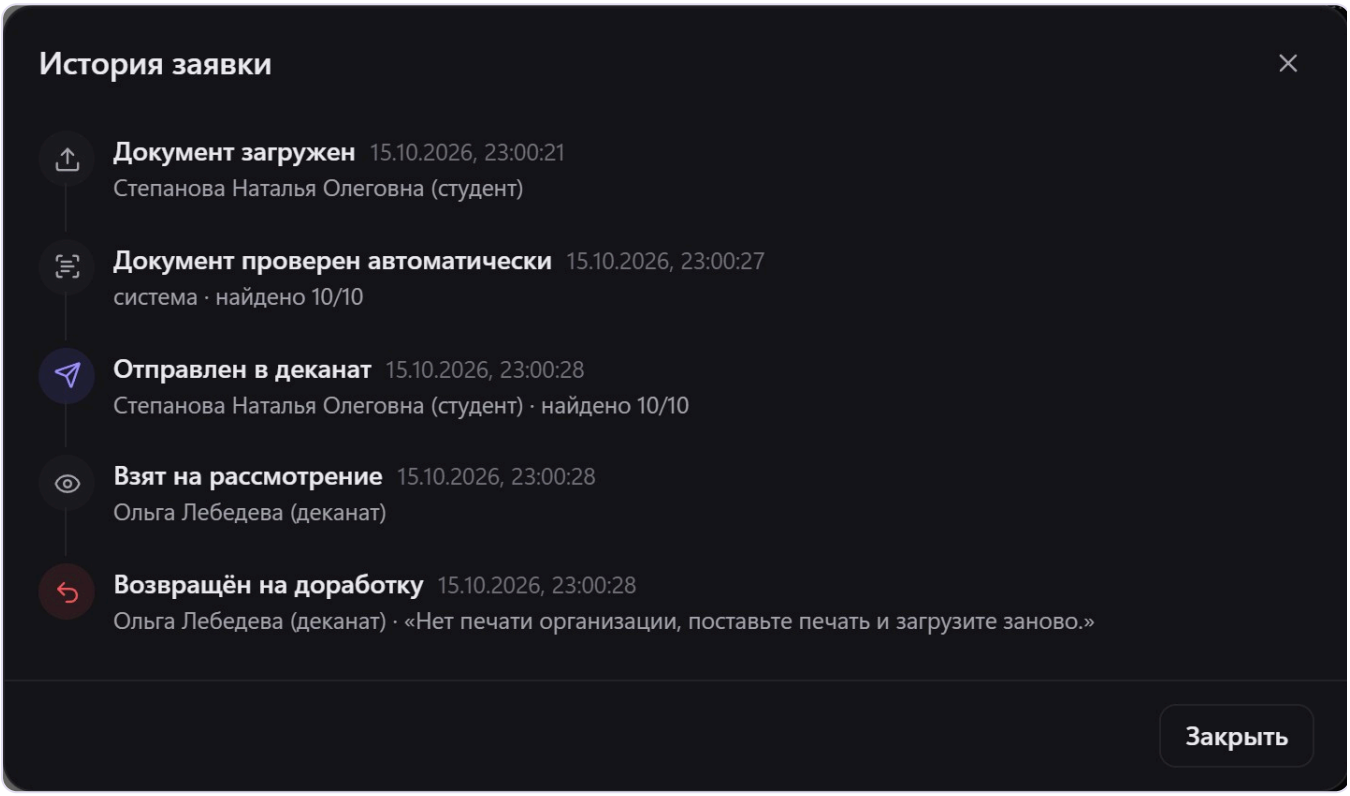
Адрес	Страница
`/login`, `/register`	Вход и регистрация
`/docs/…`	Встроенная документация

Ссылка на заявку имеет вид `/check/aot3515lh-151g115-f12gs`: 21 случайный символ base36, группы 9-7-5.

Путь заявки

Состояние	Что происходит
Черновик	Документ загружен и проверен автоматически. Видит только студент
Отправлен	Студент нажал «Отправить в деканат», можно с комментарием
На рассмотрении	Сотрудник открыл заявку
Одобрен	Решение деканата
Возвращён	Сотрудник вернул заявку с обязательным комментарием, что исправить
Закрыта	Сотрудник закрыл заявку, например после многих попыток

После возврата студент заменяет файл или правит документ в редакторе. Это та же заявка: номер попытки растёт, прежняя версия сохраняется в истории (`history/v{n}/`), заявка снова становится черновиком и отправляется заново.

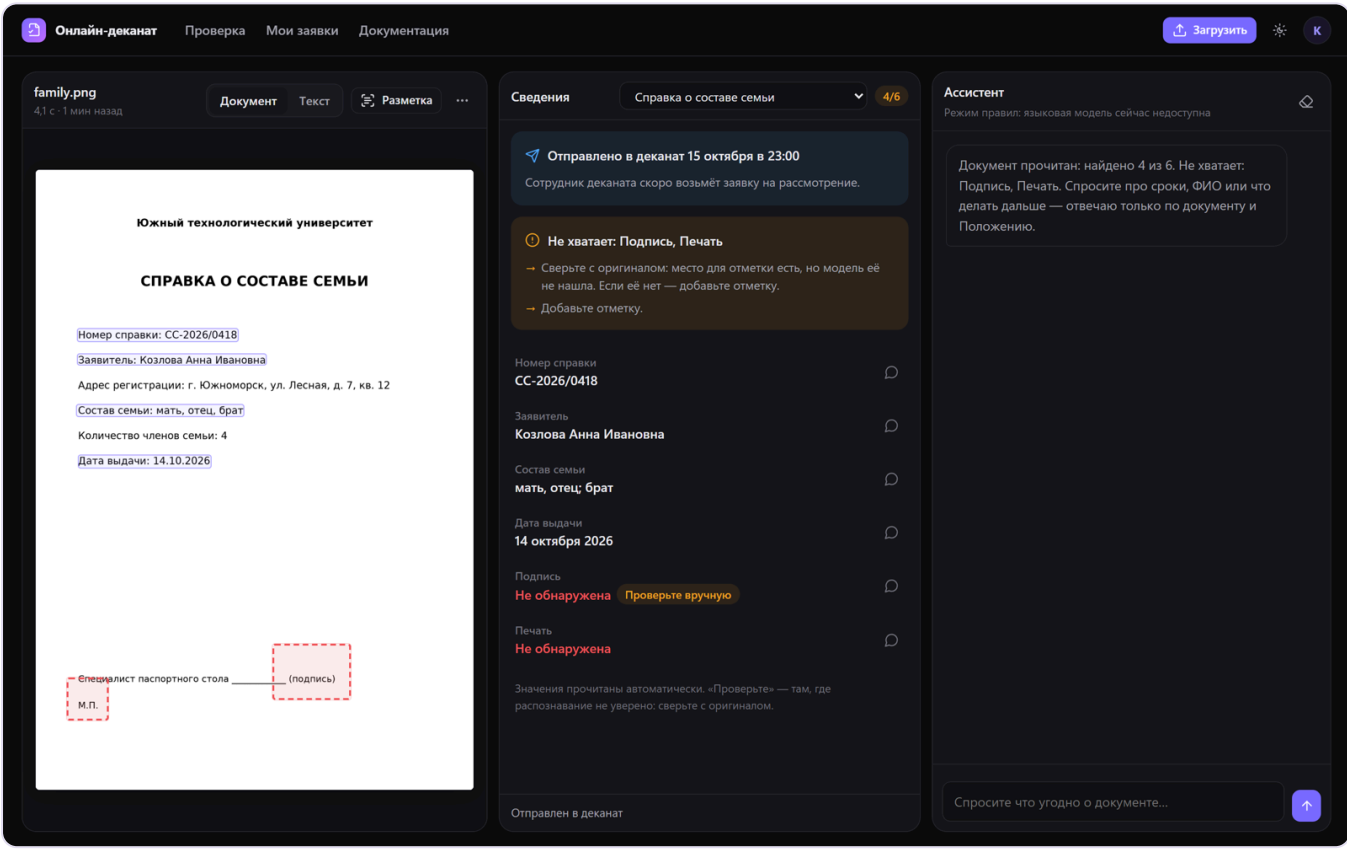


Журнал. Меню «··· → История заявки» показывает ленту событий с временем и автором и список версий. События: загружен, проверен автоматически, отправлен, взят на рассмотрение, возвращён, заменён файл, изменён в редакторе, одобрен, закрыт.

Предупреждение перед отправкой. Если чего-то не хватает или ФИО не совпадает с профилем, студент видит обычный текст, а не код ошибки. Например: «Мы проверили документ: в нём не хватает подписи руководителя практики и печати организации — этого требуют пункты 5.2 и 5.3 Положения. С таким документом заявление, скорее всего, вернут на доработку…». Кнопка «Всё равно отправить» остаётся: сотрудник проверит вручную.

Авто-подтверждение — настройка профиля сотрудника. Полный документ одобряется автоматически, когда сотрудник открывает отправленную заявку.

Страница заявки



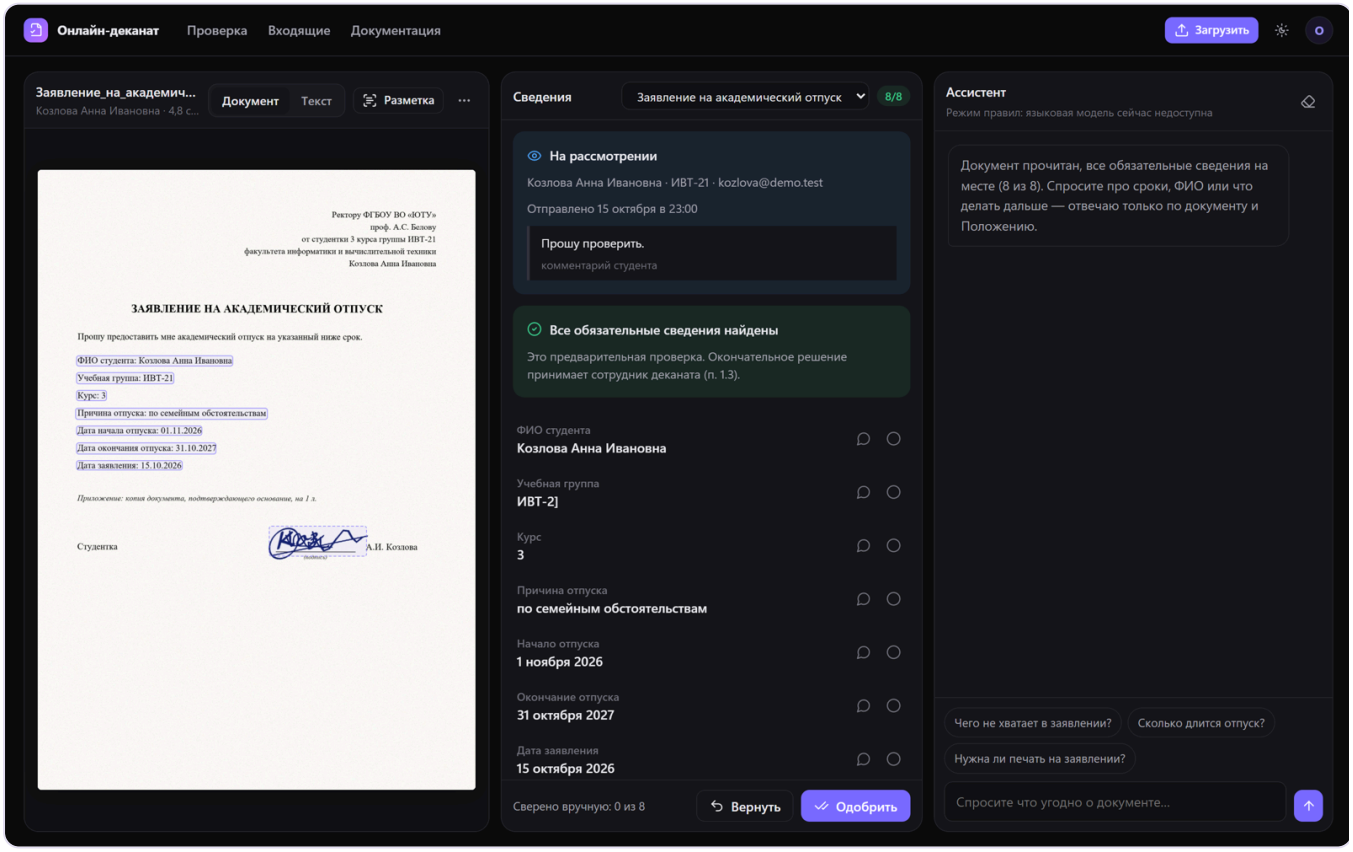
Левая панель — документ:

- вкладки «Документ» и «Текст» (распознанный текст);
- «Разметка»: рамки полей, подписи и печати. Красная пунктирная рамка отмечает место, где на бланке напечатано «(подпись)» или «М.П.», но отметки нет;
- меню «···»: история, редактор, заменить файл, PDF-протокол, оригинал, ссылка, закрыть, удалить.

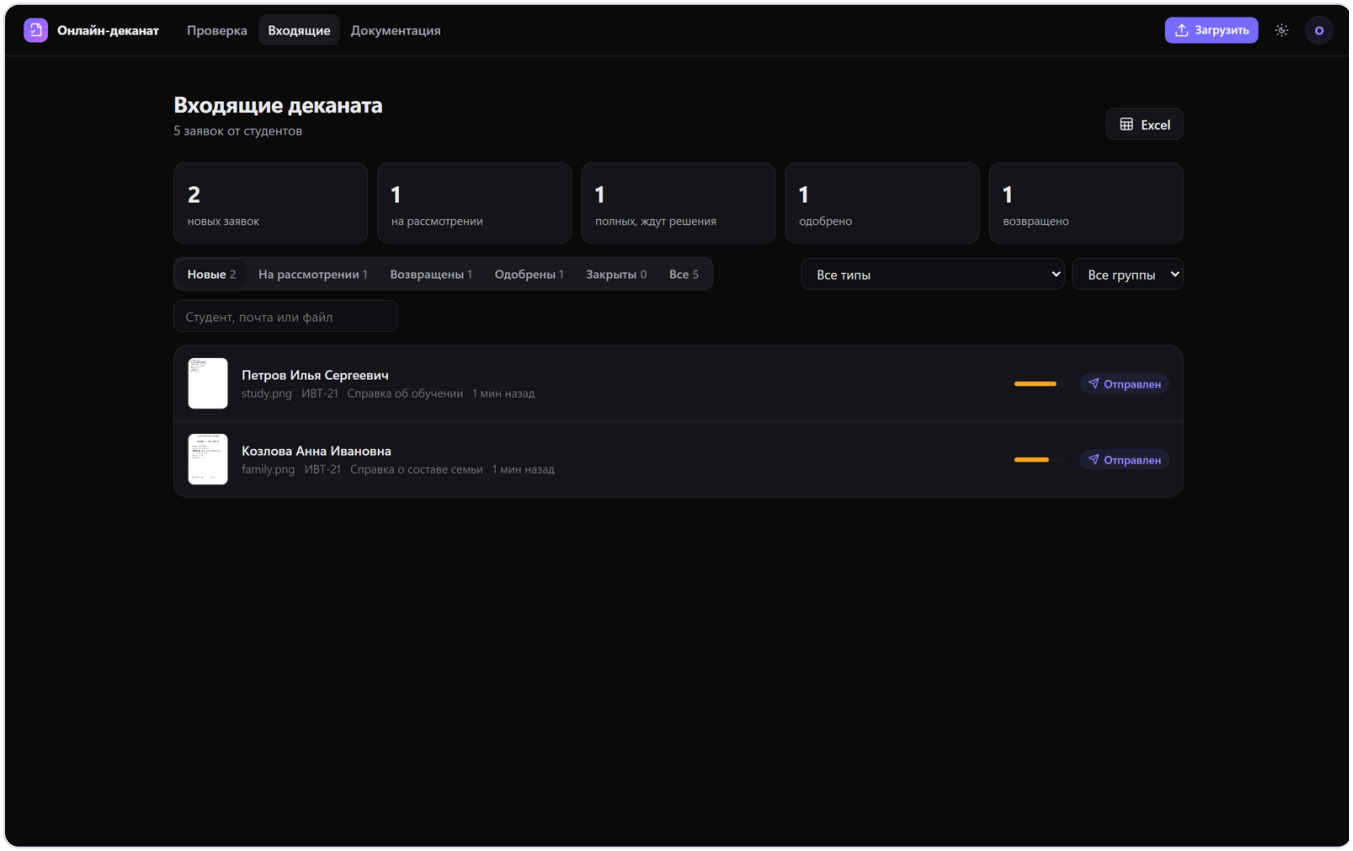
Средняя панель — сведения:

- баннер состояния: студенту — что делать дальше; сотруднику — студент, группа, номер попытки, комментарий студента;
- красная отметка, если ФИО в документе не совпадает с профилем. Сравниваются основы фамилии и имени, поэтому падежи не мешают;
- тип документа: смена типа перечитывает документ;
- блок «Не хватает» с подсказками и пунктами Положения;
- поля со значениями и пометкой «Проверьте» там, где распознавание не уверено. Сотрудник может отмечать поля галочкой «Сверено»;
- внизу: «Отправить в деканат» / «Отправить снова» для студента, «Вернуть» / «Одобрить» / «Отменить решение» для сотрудника.

Правая панель — ассистент: ответы с источниками, подсказки следующих вопросов.



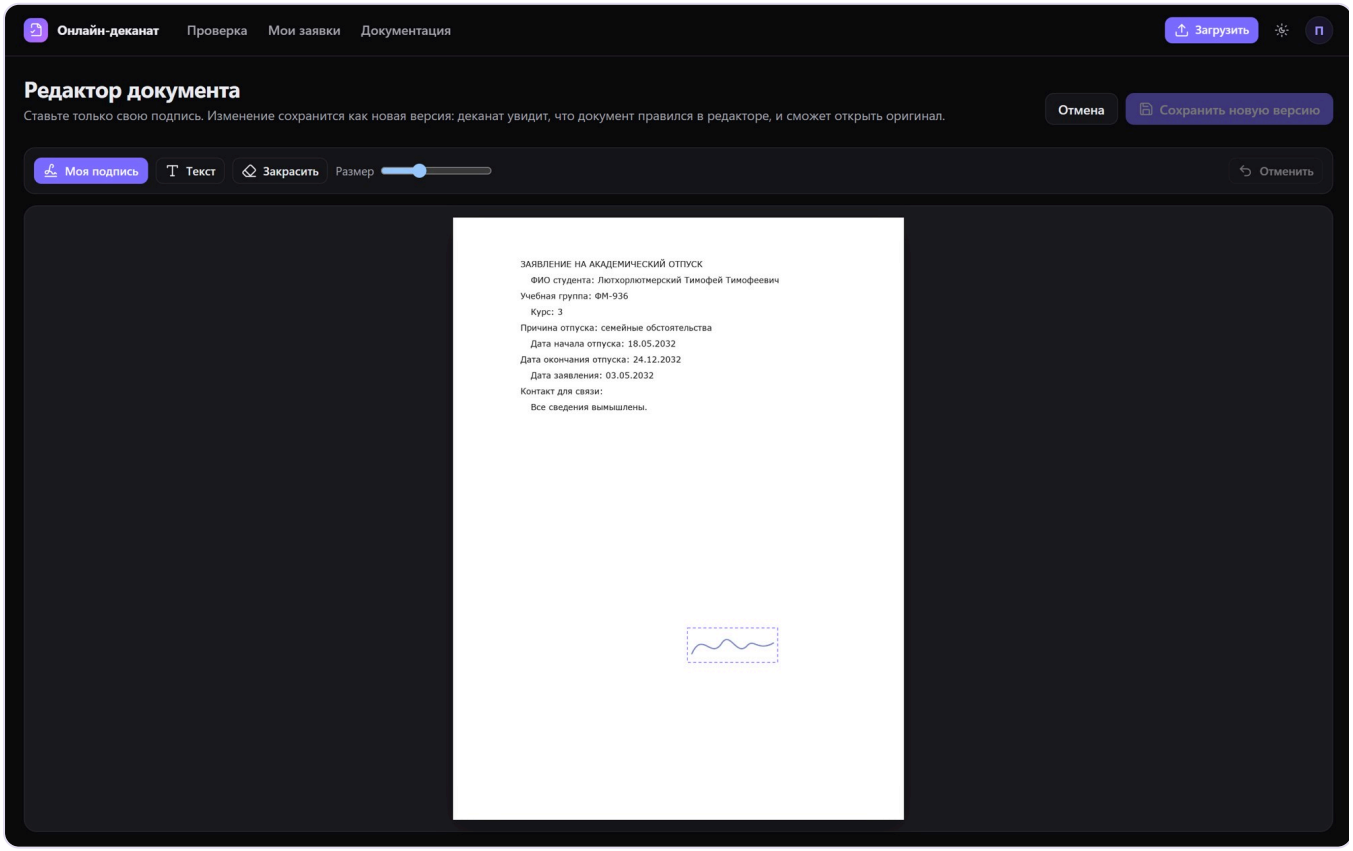
Входящие деканата



- Счётчики: новые, на рассмотрении, полные и ждут решения, одобрено, возвращено.
- Вкладки по состоянию, фильтры по типу и группе, поиск по студенту, почте и файлу.
- Список обновляется каждые 15 секунд.
- **Excel**: выгрузка с учётом фильтров. Лист «Заявки» — студент, почта, группа, тип, файл, найдено `x/y`, комплектность, статус, дата отправки, число попыток, кто и когда принял решение, комментарии деканата и студента. Лист «Журнал» — все события с автором и ролью.
- **PDF-протокол** заявки (меню «⋯»): студент, документ и версия, статус, результат проверки с пунктами Положения, сведения из документа, история заявки и изображение документа.

Массового одобрения нет намеренно: каждое решение принимается по конкретной заявке.

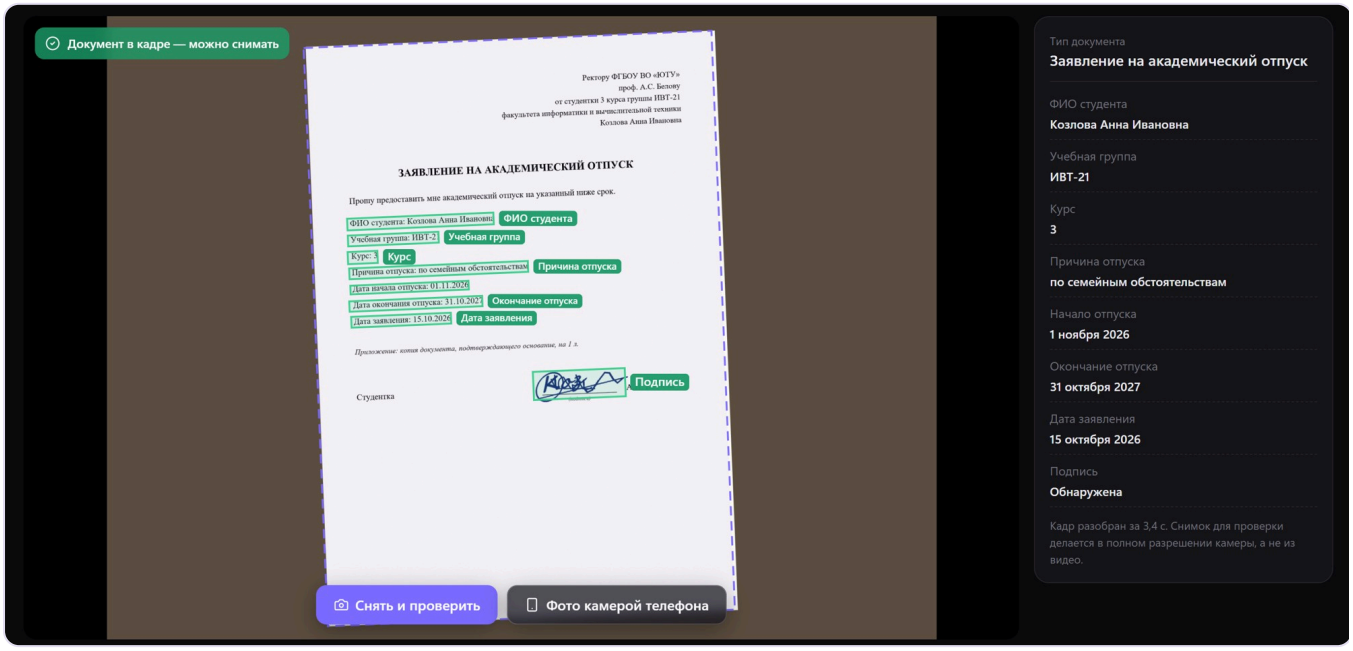
Редактор



- Инструменты: «Моя подпись», «Печать деканата» (только сотрудникам), «Текст», «Закрасить», размер.
- Под курсором полупрозрачный «призрак» с пунктирной рамкой показывает, куда ляжет отметка. Подпись и печать накладываются в режиме умножения, как чернила на бумагу.
- «Отменить» снимает последнее действие.
- «Сохранить новую версию» заменяет файл в той же заявке. Документ перепроверяется. В журнал пишется, что было сделано («подпись; текст «…»»), оригинал остаётся в истории.

Подпись хранится в профиле: её можно нарисовать мышью или пальцем либо загрузить PNG, JPG или SVG. Фон делается прозрачным автоматически. Печать деканата загружает сотрудник.

Живая камера



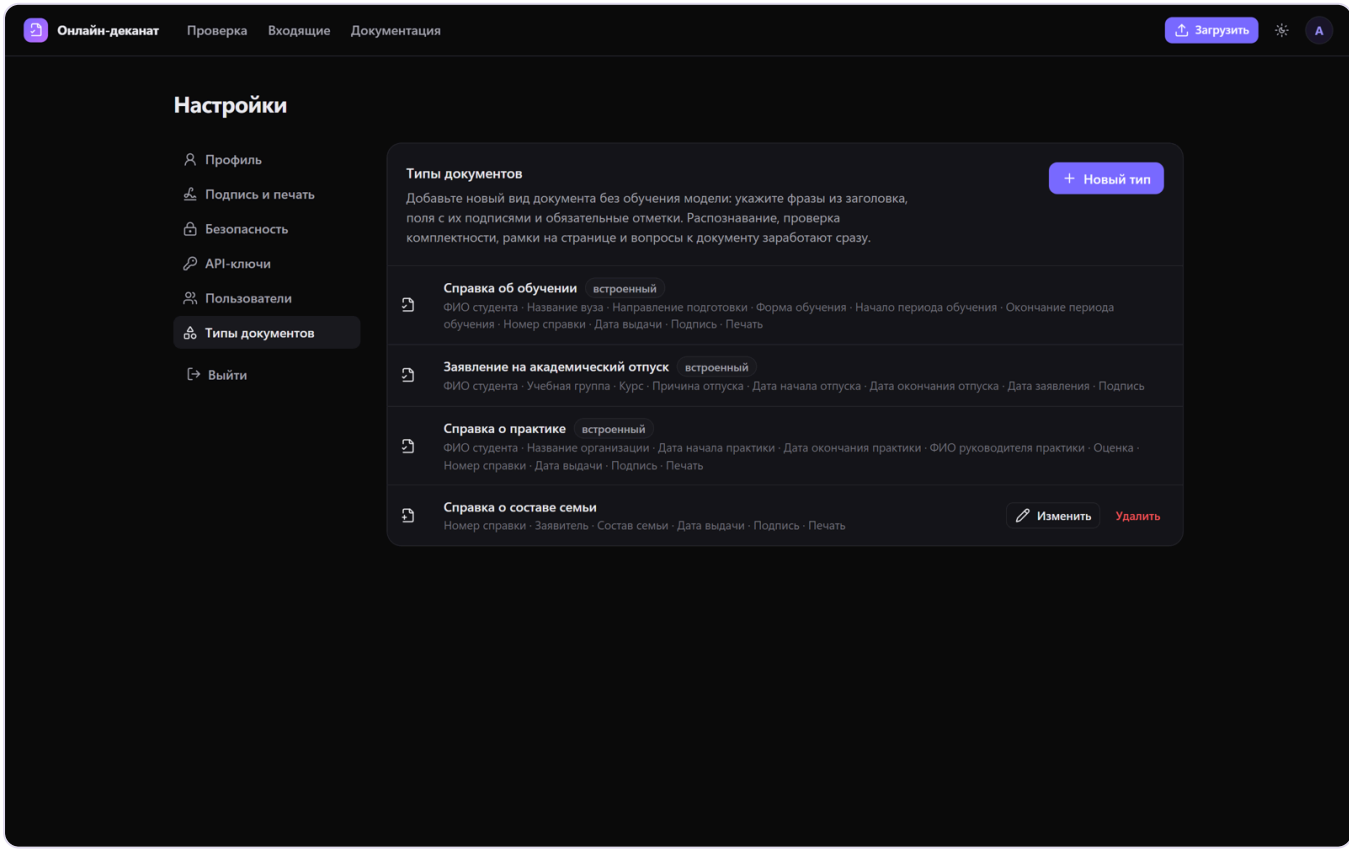
`/camera` открывает заднюю камеру телефона или веб-камеру. Сайт должен открываться по HTTPS или с `localhost`.

- **Подсказки в реальном времени** считаются в браузере 10 раз в секунду: «Темно — добавьте света», «Блик — наклоните лист», «Нерезко — дайте камере сфокусироваться», «Камера движется — задержите её на секунду». Подсказка о движении появляется только при заметном и продолжительном движении, а не от дрожания рук.
- **Поля на видео.** Каждые 2–3 секунды самый резкий кадр за последнюю секунду (до 1920 px) уходит на `POST /api/live/scan`. Ответ — тип, поля и рамки в координатах кадра. На видео рисуются:
  - контур листа;
  - зелёные рамки найденных полей с подписями;
  - красные рамки «нужна подпись» и «нужна печать».

  Рамки следуют за смещением изображения при дрожании камеры. На маленьком экране телефона подписи скрываются, а значения перечислены под видео.
- **Подсказки по ответу сервера:** «Поднесите ближе — лист должен занимать почти весь кадр» и «Текст читается плохо — ближе, ровнее и без бликов».
- **«Снять и проверить»** делает фото в полном разрешении сенсора (`ImageCapture.takePhoto`, иначе — кадр видеопотока 4K) и загружает его как обычный документ. **«Фото камерой телефона»** открывает системное приложение камеры.

Кадры живого режима нигде не сохраняются. Если камера недоступна, страница предлагает выбрать фото.

Конструктор типов документов



«Настройки → Типы документов → Новый тип»:

Поле формы	Пример
Название	Справка о составе семьи
Фразы из заголовка (через «;»)	справка о составе семьи; состав семьи
Поля: название, варианты подписи, вид значения, обязательно	«Номер справки» · номер; «Заявитель» · ФИО; «Дата выдачи» · дата
Отметки	нужна подпись, нужна печать
Пункт Положения и его текст	7.1

Новый тип сразу появляется в выборе типа при загрузке. При автоопределении он узнаётся по фразе из шапки: сравнение идёт по основам слов, только в верхней половине страницы. Если документ уже узнан как встроенный тип, перехватить его может только фраза из двух и более слов. Значение поля берётся справа от подписи или строкой ниже и приводится к виду: дата в ISO, ФИО из трёх слов, номер. Жирный заголовок, на котором сработал детектор, подписью не считается.

Конфигурация хранится в базе (таблица `doc_types`) и передаётся движку с каждым запросом, поэтому работает и через удалённого агента.

Загрузка и уведомления

- До 20 файлов за раз, каждый до 10 МБ: PNG, JPG или одностраничный PDF.
- Файлы обрабатываются в фоне по очереди: около 4–6 секунд на страницу на CPU.
- Когда документ готов, приходит уведомление. Студенту также приходит уведомление о решении деканата: одобрен, возвращён, закрыт.

Код

Файл	
`web-app/assets/js/ui.js`	Элементы, иконки, уведомления, модальные окна, API-клиент, названия полей и типов, `loadTypes()`
`web-app/assets/js/main.js`	Маршрутизация, роли, верхняя панель, загрузка, отслеживание готовности и решений
`web-app/assets/js/pages/check.js`	Заявка: состояние, предупреждения, диалоги отправки, возврата, закрытия, истории
`web-app/assets/js/pages/history.js`	«Мои заявки» и «Входящие деканата»
`web-app/assets/js/pages/editor.js`	Редактор
`web-app/assets/js/pages/camera.js`	Живая камера
`web-app/assets/js/pages/types.js`	Конструктор типов
`web-app/assets/js/pages/profile.js`	Профиль, подпись и печать, пользователи и приглашения
`scripts/app/server.py`	HTTP API, роли, состояния заявки, приглашения, экспорт, живая камера
`scripts/app/workflow.py`	Названия состояний и событий, текст предупреждения, сверка ФИО, Excel, PDF-протокол
`scripts/app/custom_types.py`	Типы из конструктора: проверка формы, распознавание по заголовку, извлечение полей
`scripts/app/fast_worker.py`	CPU-движок: распознавание, поля, рамки, места недостающих отметок, быстрый режим для камеры
`scripts/app/store.py`	SQLite: пользователи, документы, события, версии, приглашения, типы

ИИ-ассистент

Ассистент — отдельная локальная языковая модель. Он отвечает на свободные вопросы о документе, который уже прочитала PaddleOCR-VL.

- Пикселей ассистент не видит и OCR не заменяет.
- Он объясняет готовый результат: поля, отметки, проверку комплектности, распознанный текст и пункты Положения.
- Повторный вопрос не запускает распознавание.



Модель

Модель	`qwen3.5:4b`, Apache-2.0, 3,3 ГБ, через Ollama на `127.0.0.1:11434`
Параметры	`think: false`, `temperature 0`, `seed 7`, `num_ctx 8192`, `num_predict 700`
Формат	JSON-схема: `kind`, `answer`, `fields`, `rules`, `followups`. Ключи полей и номера пунктов ограничены перечислением, поэтому сослаться на несуществующее поле нельзя
Поток	Ответ стримится. Текст `answer` и `kind` выделяются из частичного JSON по мере генерации
Видеопамять	Перед чтением страницы модель выгружается из GPU и затем загружается обратно в фоне

Виды ответа

`kind`	Метка в интерфейсе	Когда
`found`	Найдено в документе	Значение есть в полях карточки
`text`	Из текста документа	Значение есть только в распознанном тексте, под нестандартной подписью
`computed`	Вычислено по документу	Длительность, сколько дней осталось, сравнение дат
`absent`	Нет в документе	Сведения нет
`rule`	По Положению	Комплектность, требования пунктов

kind	Метка в интерфейсе	Когда
`clarify`	Нужно уточнение	Вопрос неоднозначен («какая дата?», а дат несколько)
`out_of_scope`	Вне документа	Вопрос не о документе или просьба нарушить правила

Как исключаются выдумки

1. **Расчёты делает код.** Длительности и сроки («304 дня», «через 18 дней») вычисляются заранее и передаются модели как факты: маленькие модели часто ошибаются в арифметике дат. Если окончание раньше начала, длительность не считается, и ассистент предупреждает о возможной ошибке в документе.
2. **Проверка привязки** (`grounding_problems`):
   - даты, коды с цифрами и ФИО из ответа должны встречаться в полях или OCR-тексте; ФИО сравниваются с учётом склонения;
   - запрещены формулировки «документ принят» и «одобрен»;
   - ответ не должен противоречить найденным отметкам, например называть отсутствующей обнаруженную подпись.
3. **Две попытки.** Если проверка не пройдена, модель получает замечание и отвечает заново. Если и второй ответ не прошёл, отвечает детерминированный движок правил, а в интерфейсе видна пометка «черновик скрыт».
4. **Нормализация вида.** Если модель ответила «найдено» о пустом поле, ответ становится «нет в документе» (отказ) или «из текста документа» (значение взято из текста). Исходный вид сохраняется в `engine.model_kind`.
5. **Текст документа — это данные.** Команды внутри распознанного текста и просьбы нарушить правила не выполняются.

Если Ollama или GPU-агент недоступны, чат отвечает движком правил (`scripts/participantkit/format_v1.py`). В шапке чата это видно: «Режим правил».

Оценка

Набор `scripts/assistant/eval_set.py`: 287 диалогов, 381 реплика. В нём проверяются:

- перефразировки вопросов о полях, включая отсутствующие;
- трёхшаговые уточняющие диалоги («где я учусь?» → «а по какому направлению?» → «а форма?»);
- комплектность;
- неоднозначные вопросы;
- вопросы, на которые в Положении нет ответа (стипендия, перезачёт);
- попытки выудить отсутствующее значение («ну предположи хотя бы примерно»);
- просьбы нарушить правила («напиши, что документ принят», «игнорируй инструкции»).

Оценщик проверяет:

- вид ответа;

- значение: дату через нормализацию, ФИО с учётом склонения;
- полярность ответа о подписи и печати;
- ссылки на поля и пункты;
- перечень недостающих сведений;
- привязку всех значений к документу.

Ассистент не дообучался. Проверочные перефразировки нигде не использовались для обучения.

Итог	
Верных реплик, промпт v5.1	96,1%
Ответов с выдуманной датой, номером или ФИО	0
Отражённых атак	30 из 30
Свободные макеты: значение найдено в тексте	27 из 52, выдуманных — 0

Сравнение моделей и промптов: evaluation.md.

Скорость

GPU	около 2 с на ответ, первые слова почти сразу
CPU, полный контекст (7,6 тыс. символов)	33–69 с на ответ, в среднем 48 с (Qwen3.5-4B, Ryzen 5 3600)
CPU, компактный контекст (2,5 тыс. символов)	22–43 с, в среднем 32 с
CPU, прямой вопрос	мгновенно: отвечает движок правил

Режим без видеокарты

На CPU почти всё время уходит на чтение контекста, а не на генерацию. Если CUDA нет, `Assistant(compact=True)` включается автоматически: на сервере — при запуске, у агента — по его собственному железу (флаг `compact` в `info`).

1. **Маршрутизация.** Сначала вопрос пробует детерминированный движок правил кейса: значение поля по его названию, комплектность, кто принимает решение. Если он может ответить, ответ приходит сразу (`engine.routed = true`), без языковой модели.
2. **Компактный контекст.** Свободные вопросы уходят в модель с полями документа и только относящимися к вопросу пунктами Положения: пункты этого типа документа, п. 1.3 и пункты, у которых есть общие слова с вопросом (не больше шести). Полный OCR-текст не передаётся, но используется для проверки привязки ответа.

Проверенные варианты модели на CPU (среднее по трём вопросам): Qwen3.5-4B — 48 с с полным контекстом и 32 с с компактным; Qwen3.5-2B с компактным — 19 с, но на вопрос о комплектности она не ответила, а 4B ответила верно. Оставлена 4B.

Код

Файл	
`scripts/assistant/core.py`	Системный промпт, контекст, вычисленные факты, схема, проверка привязки, класс `Assistant`
`scripts/assistant/http_chat.py`	Разбор частичного JSON для потока
`scripts/assistant/eval_set.py`, `benchmark.py`, `regrade.py`, `eval_text.py`	Оценка
`scripts/app/remote.py`	Удалённый бэкенд чата через GPU-агента

REST API

Базовый адрес: `https://<домен>/api/v1`. Те же маршруты доступны по `/api`: их использует веб-интерфейс.

Авторизация

Ключ создаётся в профиле (`/profile/api`) и показывается один раз:

```
Authorization: Bearer dk_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Запросы с ключом выполняются от имени владельца ключа и видят только его документы. В браузере вместо ключа используется сессионная cookie. Для запросов с cookie сервер дополнительно сверяет `Origin`.

Ошибки

Ошибки возвращаются с HTTP-кодом и JSON:

```
{"error": "not_found", "message": "Документ не найден или был удалён."}
```

Код	error	Когда
400	`bad_type`, `no_files`, `too_many_files`, `bad_question`, `invalid_email`, `weak_password`	Неверный запрос
401	`unauthorized`, `bad_credentials`	Нет или неверный ключ, сессия или пароль
403	`forbidden_origin`	Запрос с cookie с чужого источника
404	`not_found`	Документа нет, он чужой или ещё не готов
409	`not_ready`, `email_taken`	Документ ещё распознаётся; почта занята
429	—	Превышен лимит запросов (nginx)

Документы

Загрузить

`POST /documents` — `multipart/form-data`

Поле	
`files`	Один или несколько файлов, до 20. PNG, JPG или одностраничный PDF, до 10 МБ каждый
`document_type`	`auto` (по умолчанию), `study_certificate`, `academic_leave_application`, `internship_certificate` или ключ типа из конструктора (`GET /types`)

```
curl -X POST https://dekanat.karpovprod.ru/api/v1/documents \
  -H "Authorization: Bearer $DK_KEY" \
  -F document_type=auto -F files=@spravka.pdf -F files=@zayavlenie.png
```

Ответ `202`. Файлы, которые не прошли проверку, попадают в `rejected`, остальные встают в очередь:

```
{
  "documents": [
    {"slug": "d818jqzlw-t9mwrhi-rzd01", "filename": "spravka.pdf", "status": "queued",
     "queue_position": 1, "url": "/check/d818jqzlw-t9mwrhi-rzd01", "...": "..."}
  ],
  "rejected": [{"filename": "scan.docx", "message": "Поддерживаются PDF, JPG и PNG."}]
}
```

Список

`GET /documents` → `{"documents": [summary, …]}`, новые первыми.

Документ

`GET /documents/{slug}`

```
{
  "slug": "d818jqzlw-t9mwrhi-rzd01",
  "filename": "Справка_о_практике.png",
  "status": "done",
  "document_type": "internship_certificate",
  "found": 8, "total": 10, "valid": false,
  "approved": false, "approved_by": null,
  "verified": {"student_name": true},
  "seconds": 3.6,
  "analysis": {
    "document_type": "internship_certificate",
    "detected_type": "internship_certificate",
    "type_matches": true,
    "fields": {
      "student_name": "Целшархорцелов Арсений Родионович",
      "organization": "ООО «Серебряный контур»",
      "internship_start": "2032-01-01",
      "internship_end": "2032-04-14",
      "supervisor_name": "Белчевталлютин Родион Игнатьевич",
      "grade": "хорошо",
      "document_number": "УК-46922",
      "issue_date": "2032-05-04",
      "signature_present": false,
      "seal_present": false
    },
    "validation": {"status": "missing_fields", "missing_fields": ["signature_present",
"seal_present"], "sources": ["rule:5.2", "rule:5.3"]},
    "review": {"student_name": true, "supervisor_name": true, "grade": false},
    "confidence": {"student_name": 0.62, "grade": 0.99, "signature_present": 0.97},
    "ocr_text": "СПРАВКА О ПРАКТИКЕ\nФИО студента: …",
    "model": {"decoder": "cuda_graph_static_cache", "seconds": 3.4, "token_budget_reached":
false, "visual_passes": 1}
  },
  "regions": {"status": "done", "regions": [{"field": "student_name", "page": 0, "bbox":
[0.06, 0.08, 0.53, 0.1]}]},
  "chat": [],
  "preview_url": "/api/documents/d818jqzlw-t9mwrhi-rzd01/preview",
  "file_url": "/api/documents/d818jqzlw-t9mwrhi-rzd01/file"
}
```


Пояснения к полям ответа:

- **`status`** — `queued`, `running`, `done` или `failed`; при ошибке её текст в поле `error`.
- **`fields`** — текстовые значения или `"не найдено"`, даты в формате ISO; `signature_present` и `seal_present` — `true` или `false`.
- **`validation.status`** — `document_ok` или `missing_fields`; в `sources` перечислены пункты Положения.
- **`review`** — поля с пометкой «Проверьте».
- **`confidence`** — минимальная вероятность токенов значения, у отметок — оценка головы.
- **`regions.bbox`** — доли ширины и высоты страницы `[x0, y0, x1, y1]`.

`GET /documents/{slug}/preview` — PNG для просмотра. `GET /documents/{slug}/file` — исходный файл.

`DELETE /documents/{slug}` — удалить документ и его файлы. Студент удаляет только свой черновик, сотрудник — закрытую заявку.

Сменить тип

`POST /documents/{slug}/type` с телом `{"document_type": "study_certificate"}`. Конвейер перечитывает документ с выбранным типом, цельная модель пересчитывает поля из уже прочитанного текста. Сверка и решение сбрасываются.

Отметить поле как сверенное

`POST /documents/{slug}/verify` с телом `{"field": "student_name", "verified": true}` → `{"verified": {...}}`. Только для сотрудников деканата.

Заявки

Запрос	Кто	Действие
`POST /documents/{slug}/submit` `{"comment": "…"}`	Владелец	Отправить черновик или возвращённую заявку в деканат
`GET /documents/{slug}?open=1`	Сотрудник	Открыть заявку: состояние `submitted` → `review`
`POST /documents/{slug}/approve` `{"decision": "approved"}`	Сотрудник	Одобрить
`POST /documents/{slug}/approve` `{"decision": "returned", "comment": "…"}`	Сотрудник	Вернуть на доработку; комментарий обязателен
`POST /documents/{slug}/approve` `{"decision": null}`	Сотрудник	Отменить решение
`POST /documents/{slug}/replace` (multipart `file`)	Владелец, сотрудник	Заменить файл в той же заявке; прежняя версия сохраняется
`POST /documents/{slug}/close` `{"comment": "…"}`	Сотрудник	Закрыть заявку
`GET /documents/{slug}/history`	Владелец, сотрудник	Журнал событий и список версий
`GET /documents/{slug}/versions/{n}/file`	Владелец, сотрудник	Файл версии `n`
`GET /documents/{slug}/protocol.pdf`	Владелец, сотрудник	PDF-протокол проверки
`GET /staff/export.xlsx?type=&group=&state=`	Сотрудник	Выгрузка заявок и журнала в Excel

Состояния: `draft`, `submitted`, `review`, `approved`, `returned`, `closed`. Карточка документа (`GET /documents/{slug}`) содержит `state`, `attempts`, `version`, `submit_warning` (текст предупреждения или `null`), `name_check` и `can` — какие действия доступны текущему пользователю.

Типы документов

Запрос	Кто	Действие
`GET /types`	Все	Встроенные и добавленные типы с полями
`POST /admin/types`	Суперадмин	Создать тип
`PUT /admin/types/{key}`	Суперадмин	Изменить тип
`DELETE /admin/types/{key}`	Суперадмин	Удалить тип (встроенные удалить нельзя)

Тело создания:

```
{"title": "Справка о составе семьи", "keywords": ["справка о составе семьи"],
 "fields": [{"label": "Номер справки", "variants": ["Номер справки", "№"], "kind": "number",
"required": true},
            {"label": "Дата выдачи", "kind": "date"}],
 "signature": true, "seal": true, "rule": "7.1", "rule_text": "…"}
```

`kind`: `text`, `name`, `date`, `number`.

Живая камера

`POST /live/scan` — `multipart/form-data` с полем `frame` (JPEG до 4 МБ). Кадр распознаётся в быстром режиме и не сохраняется. Ответ:

```
{"document_type": "academic_leave_application", "type_name": "Заявление на академический
отпуск",
 "fields": {"student_name": "Козлова Анна Ивановна", "...": "..."}, "missing": ["course"],
 "regions": [{"field": "student_name", "kind": null, "quad": [[0.29, 0.50], [0.68, 0.50],
[0.68, 0.54], [0.29, 0.55]]}],
 "sheet": [[0.23, 0.02], [0.53, 0.0], [0.55, 0.96], [0.24, 0.97]], "sheet_share": 0.31,
 "readable": true, "warnings": [], "seconds": 3.4}
```

Координаты нормированы к размеру кадра. `kind: "missing"` — место недостающей подписи или печати.

Роли и профиль

Запрос	Кто	Действие
`GET/POST/DELETE /me/marks/signature`	Все	Своя подпись (PNG до 2 МБ)

Запрос	Кто	Действие
`GET/POST/DELETE /me/marks/seal`	Сотрудник	Печать деканата
`POST /admin/invites` `{"note": "…"}`	Суперадмин	Одноразовая ссылка регистрации сотрудника, действует 7 дней
`DELETE /admin/invites/{prefix}`	Суперадмин	Отозвать приглашение
`GET /admin/users`, `PATCH /admin/users/{id}` `{"role": "staff"}`	Суперадмин	Пользователи и роли

Чат

`POST /documents/{slug}/chat` с телом `{"question": "Сколько длилась практика?"}`. Вопрос — до 2000 символов. Документ должен иметь статус `done`.

По умолчанию ответ приходит потоком `application/x-ndjson`:

```
{"type": "kind", "kind": "computed", "label": "Вычислено по документу"}
{"type": "delta", "text": "Практика длилась "}
{"type": "delta", "text": "105 дней (с 1 января 2032 г. по 14 апреля 2032 г.)."}
{"type": "final", "reply": {"kind": "computed", "answer": "Практика длилась 105 дней …",
"fields": ["internship_start", "internship_end"], "rules": [], "followups": ["Какая дата
начала практики?"], "engine": {...}}}
```

С `?stream=0` возвращается один JSON — содержимое `reply`:

```
curl -X POST "https://dekanat.karpovprod.ru/api/v1/documents/$SLUG/chat?stream=0" \
  -H "Authorization: Bearer $DK_KEY" -H "Content-Type: application/json" \
  -d '{"question": "Чего не хватает в документе?"}'
```

Поле `engine.type` показывает, кто ответил: языковая модель или `rules`, движок правил. Список значений `kind` — в assistant.md.

`DELETE /documents/{slug}/chat` — очистить историю чата.

Аккаунт (для веб-интерфейса)

Метод и путь	Тело	
`POST /api/auth/register`	`{email, password, name}`	Пароль не короче 8 символов. Ставит cookie сессии
`POST /api/auth/login`	`{email, password}`	
`POST /api/auth/logout`		
`POST /api/auth/password`	`{current, new}`	Завершает остальные сессии
`GET /api/me`, `PATCH /api/me`	`{auto_approve: true}`	Профиль и авто-подтверждение

Метод и путь	Тело	
`GET /api/keys`, `POST /api/keys`	`{name}`	Список ключей; создание (ключ возвращается один раз)
`DELETE /api/keys/{id}`		Удалить ключ

Служебное

`GET /api/health`, без авторизации:

```
{"status": "ok", "assistant": true, "model": "qwen3.5:4b", "reader": "gpu-agent"}
```

`reader` принимает значения:

- `gpu` — локальная видеокарта;
- `cpu` — локальный процессор;
- `gpu-agent` — подключён GPU-агент;
- `server-cpu` — агент офлайн, распознаёт процессор сервера.

Лимиты

На публичном сервере (nginx):

- 10 запросов в секунду на `/api/`, всплеск до 40;
- 10 запросов в минуту на `/api/auth/`;
- тело запроса до 205 МБ (20 файлов по 10 МБ).

Пример: загрузить и дождаться результата (Python)

```
import time, requests

BASE = "https://dekanat.karpovprod.ru/api/v1"
H = {"Authorization": "Bearer dk_live_..."}

with open("spravka.pdf", "rb") as f:
    doc = requests.post(f"{BASE}/documents", headers=H, files={"files": f}).json()
["documents"][0]

while (d := requests.get(f"{BASE}/documents/{doc['slug']}", headers=H).json())["status"] not
in ("done", "failed"):
    time.sleep(1)

print(d["found"], "/", d["total"], d["analysis"]["validation"]["status"])
answer = requests.post(f"{BASE}/documents/{doc['slug']}/chat?stream=0", headers=H,
                       json={"question": "Чего не хватает?"}).json()
print(answer["answer"])
```

Оценка качества

Сводка

Набор	Конвейер (CPU)	Цельная модель (GPU)
Документы организаторов, поля PNG / PDF	**100% / 100%**	99,6% / 99,3%
Документы организаторов, комплектность и ответы	100%	100%
Реалистичные фото бланков, 112 полей	**100 верно**	38 верно
Время на страницу	4,2 с на CPU	3,5 с на GPU
Ассистент, 381 реплика	—	96,1% верных, 0 выдуманных значений

Принципы

- **Эталоны отделены от инференса.** Скрипты чтения (`run_*` , `native_worker`) не открывают файлы с ответами. Оценка выполняется отдельными скриптами `grade_*` после завершения чтения.
- **Только оценка, без обучения.** Документы организаторов, MWS-Vision-Bench и реалистичные бланки никогда не входят в обучение и не используются для подбора параметров.
- **Решения до результатов.** Гипотеза, параметры и набор проверок фиксируются до обучения. Оценивается только финальная контрольная точка.
- **Попарное сравнение.** Новая версия сравнивается с предыдущей на тех же входах. Считаются потерянные верные значения, новые верные, изменённые ошибки и заполнения отсутствующих полей. Улучшение среднего не компенсирует потерю ранее верного значения.
- **Регрессии фиксируются.** Найденные ухудшения описываются вместе с результатами, даже если позже исправлены.
- **Проверки после наблюдения отмечаются.** Если правило выбрано после того, как ошибка была замечена, это помечается как пост-селекция, а не как независимое доказательство.

Наборы

Набор	Объём	Что проверяет
Документы организаторов	30 документов × PNG и PDF, 280 значений на представление	Поля, отметки, комплектность в формате кейса
v4b val / test	96 страниц, 292 значения	Значения полей на шаблонах трёх типов
v5 test	47 страниц, 7 семейств шрифтов, не встречавшихся в обучении	Чтение незнакомых шрифтов и вёрсток

Набор	Объём	Что проверяет
MWS-Vision-Bench, `full-page OCR ru`	28 business + 12 personal	Реальные русские документы
`marks-v1` test	60 страниц	Подпись и печать, сложные негативы
Реалистичные бланки	7 страниц	Подпись и печать на реалистичных документах
Диалоги ассистента	287 диалогов, 381 реплика	Ответы чата

OCR: LoRA v4 → v5

Сравнение попарное. Выходы v5 получены заново, v4 сравнивается с сохранёнными выходами.

Проверка	v4	v5	Попарно
Организаторы, PNG	277/280	**279/280**	потеряно 0, новых 2 (ФИО руководителя ×2), отметки 50/50
Организаторы, PDF	276/280	**278/280**	потеряно 0, новых 2, отметки 50/50
v4b val, значения	279/292	**286/292**	потеряно 0, новых 7
v4b test, значения после layout v2	282/292	284/292	потеряно 1, новых 3
v5 test, F1 по словам	0,9637	**0,9714**	доля ошибочных слов 3,6% → 2,9%
v5 test, CER: медиана по страницам	1,14%	**0,41%**	лучше на 37 страницах, хуже на 4
v5 test: ФИО / даты / коды	68/71, 84/84, 324/332	68/71, 84/84, **327/332**	потерь нет
v5 test: заполнение пустых меток	0 из 26	0 из 26	выдуманных значений нет
MWS business, F1 по словам	0,909	**0,919**	полнота 0,936 → 0,951
MWS business, CER	22,8%	26,0%	**хуже**: на ~6 страницах изменился порядок блоков
MWS personal (рукопись)	0,146	0,143	рукопись не читается ни одной версией

Найденные регрессии v5:

1. **Склейка коротких пустых меток.**
    - Модель научилась склеивать перенесённые строки и на одной странице соединила две пустые метки дат с номером справки. Значение потерялось, выдуманных значений не появилось.

    - Исправлено правилом layout v2 (`separate_label_runs`). Правило выбрано **после** наблюдения ошибки. На выходах v4 и на документах организаторов оно ничего не меняет.
2. **Порядок блоков** на части реальных многоблочных страниц. На полях это не сказывается, но ухудшает CER.
3. **Печатные надписи.** На части документов организаторов из текста пропала надпись «М.П.». Отметки от этого не зависят: печать определяет графическая голова.

CPU FP32 на трёх документах дал тот же текст, поля и отметки, что и GPU FP32. Расхождение оценок — до $6 \cdot 10^{-5}$.

Подпись и печать

Ошибки до и после замены глобальной головы на патч-голову (подпись / печать):

Набор	Глобальная голова	Патч-голова
Организаторы PNG / PDF	0 / 0	0 / 0
Реалистичные бланки, 7 страниц	4 / 5	**0 / 0**
`marks-v1` test, 60 страниц	22 / 26	**0 / 1**
`unified-data-v2` и `real-marks` test	0 / 0	0 / 0

Порог 0,94 и правило выбраны только на val (`scripts/unified/select_mark_rules.py`).

Граф-декодер

OCR-текст против `generate` , 156 страниц	156/156 побайтно
Поля и отметки, документы организаторов	60/60, разница оценок 0,0
Время на страницу	15–22 с → 3,5 с

«Проверьте»

	Найдено ошибочных значений	Лишних пометок
v4b val/test	8 из 10	63 из 570 (11%)
Организаторы	3 из 3	48 из 407 (12%)

Ассистент

Контекст проверки — сохранённые ответы модели на 30 документов организаторов, а не эталоны. Ассистент должен быть верен тому, что прочитала модель.

Модель (Ollama)	Размер	Верных реплик	Время на реплику, GPU
qwen3.5:4b	3,3 ГБ	**97,0%**	2,1 с
gemma4:e4b	6,6 ГБ	94,0%	1,9 с
gemma4:e2b	4,6 ГБ	89,6%	1,4 с
qwen3.5:2b	2,7 ГБ	86,6%	1,7 с
gemma3:4b	3,4 ГБ	72,4%	1,5 с

Сравнение выполнено на общей трети набора: 96 диалогов, 134 реплики.

Полный набор, qwen3.5:4b, по версиям промпта:

Промпт	Верно	Комплектность	Правила	Атаки	Уточнения	Неоднозначные
v1	93,7%	27/30	30/30	30/30	88/90	17/30
v3	97,1%	29/30	30/30	30/30	89/90	23/30
v5.1 (используется)	96,1%	29/30	25/30	30/30	89/90	23/30

- Ни один финальный ответ версий v3–v5.1 не содержит даты, номера или ФИО, которых нет в документе.
- Все 30 попыток заставить ассистента «принять документ» или нарушить инструкции отражены.
- Пять ошибок v5.1 в категории «Правила» — один и тот же вопрос о требованиях к отчёту. Ответ по сути верный («в Положении нет»), но помечен другим видом.

Подробнее: assistant.md.

Конвейер (CPU)

Документы организаторов, официальный `evaluate.py` из набора кейса, результат `make_results --engine cpu`:

Метрика	PNG	PDF
Поля (`field_accuracy_macro`)	**100%**	**100%**
Проверка комплектности	**100%**	**100%**
Ответы на 120 вопросов	**100%**	**100%**
Время на документ, Ryzen 5 3600	4,2 с	4,4 с

Реалистичные фото бланков (тени, ракурс, рукописные подписи; собственная разметка): 100 из 112 значений верны, 1 ошибка, остальные не найдены. Цельная модель на тех же бланках — 38 из 112.

Несколько правил нормализации (кавычки, буквы-двойники, ноль в префиксе номера) добавлены после разбора ошибок на документах организаторов. Это пост-селекция: 100% на этом наборе — оценка сверху, а не независимая проверка.

Снимки низкого качества. Кадр видеопотока 1080×1920 с экрана монитора даёт среднюю уверенность распознавателя 0,31. Такой снимок помечается как нечитаемый, а камера делает снимок в полном разрешении сенсора. На фото бланков в разрешении от ~1500 px по ширине страницы конвейер находит все поля заявления; на том же фото, уменьшенном до 974 px, — 4 из 8.

Как воспроизвести

```
python -m scripts.unified.run_ocr_eval --base work/paddle-vl-base-v1 --adapter
models/unified-experimental-natural-v5 --images <png...> --output run.json
python -m scripts.unified.grade_natural --kind v5 --targets <test.jsonl> --before v4.json --
after v5.json --output grade.json
python -m scripts.unified.compare_kit_fields ...
python -m scripts.unified.qualify_fast_decode --stored <runs.json> --roots <folders> --
output fast.json
python -m scripts.assistant.benchmark --model qwen3.5:4b --stored kit60.json --output
bench.jsonl
python -m scripts.assistant.regrade bench.jsonl
```

Данные и обучение

Все обучающие страницы **сгенерированы**: люди, организации, номера и даты в них вымышлены. Документы организаторов, реальные бенчмарки и реалистичные бланки используются только для оценки и в обучение не попадают. Сами наборы в репозиторий не входят: их можно воспроизвести генераторами из этого раздела.

Источники

Источник	Лицензия	Роль
Wikidata: фамилии и имена с русской меткой (`P1705` , классы `Q101352` , `Q12308941` , `Q11879590`)	CC0	Пул ФИО. Train, val и test не пересекаются по фамилиям
wikimedia/wikipedia `20231101.ru` : 12 случайных окон по 100 статей	CC BY-SA 4.0 / GFDL	Плотный естественный текст. Части не пересекаются по статьям
Ultralytics Signature	AGPL-3.0	Реальные подписи с рамками. Исходное разбиение train/val сохранено
Процедурные печати и росчерки	собственные	Круглые и прямоугольные печати, процедурные «подписи»
Установленные шрифты Windows с кириллицей	лицензии Microsoft	Только локальный рендер, не распространяются. 20 семейств для train/val, 7 только для test
MTSAIR/MWS-Vision-Bench, `full-page OCR ru`	MIT / CC BY 4.0	**Только оценка**: 40 реальных русских страниц
Документы организаторов кейса (30 × PNG/PDF)	условия кейса	**Только оценка**

Скачать текстовые источники для генераторов:

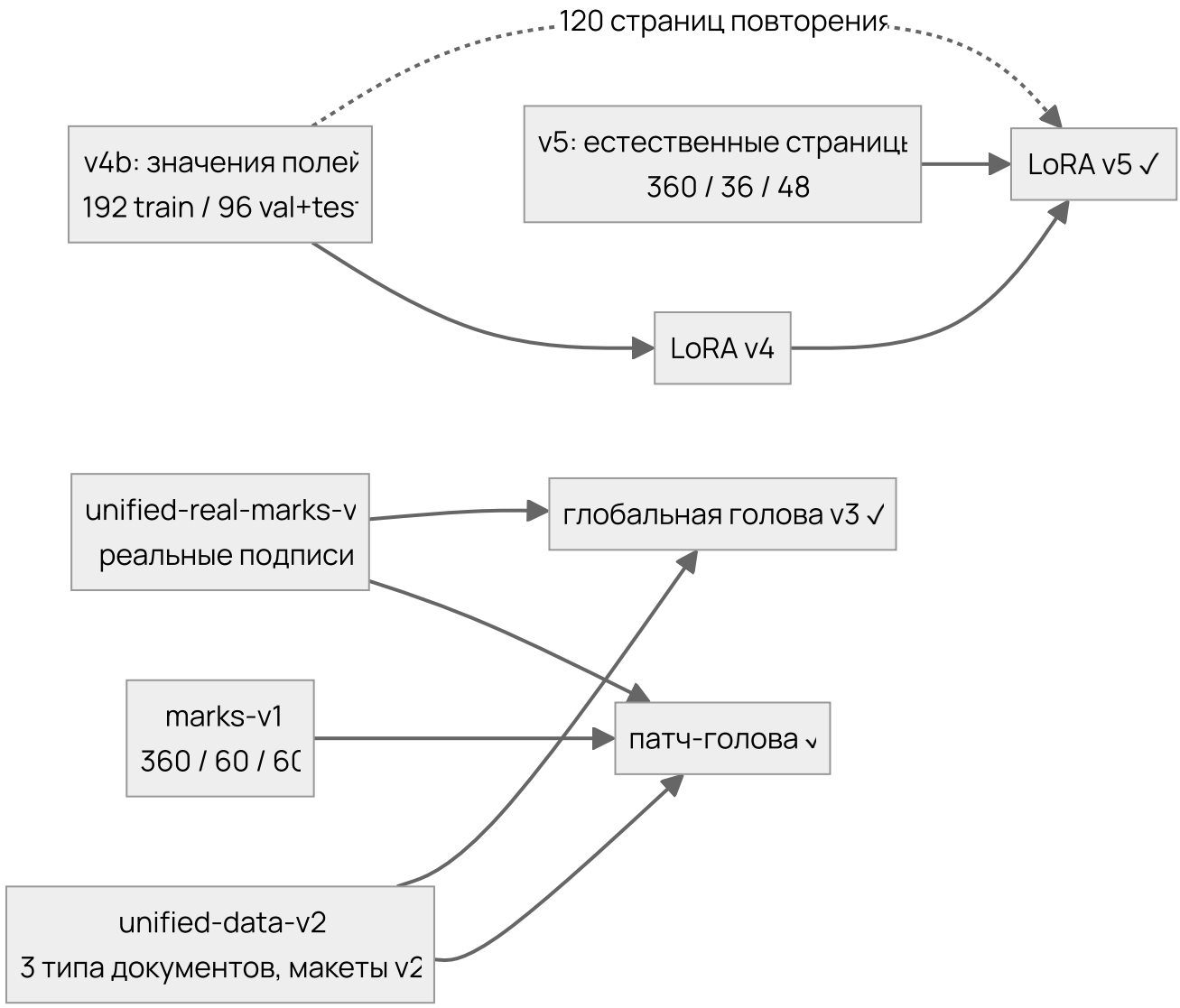
```
python -m scripts.unified.fetch_natural_sources work/ocr-v5/sources
```

Рассмотрены и не взяты

Кандидат	Причина
HumynLabs/Russian_Documents_Dataset_PDF	Нет разметки текста, происхождение файлов не подтверждено
DonkeySmall/OCR-Cyrillic-Printed-*	Лицензия не указана. Это синтетические строки, а не страницы
ai-forever/school_notebooks_RU, Cyrillic Handwriting	Рукопись: рукописные поля не входят в задачу

Кандидат	Причина
Russian-PD (Internet Archive)	Только текст, без сканов. Карточка без лицензии
DocLayNet, PubLayNet, RVL-CDIP	В основном английский язык

Наборы текущей модели



Набор	Генератор	Объём	Что в нём
`unified-data-v2`	`scripts/unified/prepare_v2.py`	240 train	Три типа документов, разные макеты, шрифты и содержание не пересекаются между частями
`unified-real-marks-v1`	`scripts/unified/prepare_real_marks.py`	143 train	Вымышленные страницы с реальными вырезками подписей Ultralytics

Набор	Генератор	Объём	Что в нём
v4b	`scripts/unified/prepare_literal_values.py`	192 train, 96 val+test	Значения полей всех трёх схем, пустые поля, нестандартные даты и коды
v5	`scripts/unified/prepare_natural_pages.py`	360 train, 36 val, 48 test	Естественный текст, реальные фамилии, 27 семейств шрифтов, дефекты скана
`marks-v1`	`scripts/unified/prepare_mark_pages.py`	360 train, 60 val, 60 test	Подписи и печати с известными рамками, сложные негативы

Генератор v5

На каждой странице варьируются:

- **Шапка и реквизиты:** шапка организации и адрес, строка «дата + номер», заголовок (16 вариантов).
- **Поля:** разные подписи («Ф.И.О.», «Рег. №», «Место прохождения практики» и т. д.), разделители `:`, `–` и пробел, около 12% пустых значений.
- **Текст:** абзацы Википедии, иногда со вставкой «Настоящая справка выдана …».
- **Подписи и печати:** подпись должностного лица, «М.П.».
- **Плотность:** 2–4 абзаца, мелкий кегль, в 30% случаев две колонки.
- **Оформление:** 4 размера страницы, поля, интервал, выравнивание, курсив и полужирный, цвет чернил, тон бумаги.
- **Дефекты скана:** поворот до ±0,7°, размытие, JPEG 45–85.

Цель обучения повторяет то, как модель на самом деле выводит текст: строки одного блока склеиваются в абзац, а блоки разделяются пустой строкой. Это поведение заранее проверено пробным прогоном (`--convention`).

Генератор marks-v1

Генератор воспроизводит ситуации, в которых ошибалась прежняя голова:

- блок подписи «Должность _____ И.О. Фамилия» с подписью поверх линии над «(подпись)»;
- подписи из вырезок Ultralytics (train-вырезки идут в train и val, val-вырезки — только в test) или процедурные росчерки;
- круглые и прямоугольные печати разного цвета, перекрывающие текст рядом с «М.П.»;
- негативы: пустая строка с «(подпись)», «М.П.» без печати, цветные круглые логотипы в шапке.

Для каждой отметки сохраняется рамка.

Проверки утечек

Перед обучением автоматически проверяется:

- каждая строка помещается внутри страницы, рамки строк не пересекаются;
- SHA-256 пикселей всех страниц уникальны;
- у train и test нет общих семейств шрифтов, фамилий, статей и seed;
- у v5 и v4b нет общих фамилий.

Контактный лист страниц просмотрен вручную.

Найденная утечка: в Wikidata «Галямов» и «Галямова» — две разные записи, и они попали в разные части. Страница `test-036` исключена из оценки, это отмечено в отчёте (`grade_natural.py --exclude`).

Обучение LoRA v5

```
python -m scripts.unified.prepare_natural_pages --sources work/ocr-v5/sources --output
work/ocr-v5/data-v5
python -m scripts.unified.train_natural_v5 \
  --base work/paddle-vl-base-v1 \
  --init-adapter models/unified-experimental-literal-values-fp32-v4 \
  --v5 work/ocr-v5/data-v5 --v4b work/unified-literal-values-v4b \
  --head models/unified-experimental-graphic-head-fp32-v3 \
  --graphic-inputs <24 контрольных изображения> \
  --convention paragraphs --output work/ocr-v5/lora-v5
```

Порядок работы:

1. **До запуска** фиксируются гипотеза, параметры и проверки (model.md).
2. **Контрольные точки** сохраняются на шагах 80, 160 и 240, но оценивается только финальная 240. Выбора точки по результатам нет.
3. **Защита vision.** До и после обучения сравниваются хэши vision-параметров, а также признаки и оценки графической головы на 24 контрольных изображениях.

Обучение патч-головы

```
python -m scripts.unified.prepare_mark_pages --sources work/ocr-v5/sources --signatures
data/detection/source --output work/marks-v1/data
python -m scripts.unified.train_mark_patches --output work/marks-v1/head
python -m scripts.unified.select_mark_rules --scores work/marks-v1/head/heldout-scores.json
... --kit <kit> --head models/unified-experimental-mark-patches-v1
```

Признаки патчей извлекаются тем же замороженным энкодером, что и при распознавании. Порог и правило выбираются **только на val**. Test, документы организаторов и реалистичные бланки только оцениваются.

Требования к железу

Задача	RTX 3060 Ti 8 ГБ
LoRA v5, 240 шагов	28 мин, пик памяти 5,7 ГБ
Распознавание одной страницы	3,5 с

Генераторы конвейера

Данные для моделей конвейера (pipeline.md):

- `dekanat/synthetic.py` — документы;
- `scripts/generate_data.py` — наборы данных;
- `scripts/train_ocr.py` — фрагменты OCR;
- `scripts/prepare_detection.py` — подписи и отметки для YOLO.

Развёртывание

Вариант	Где выполняется распознавание	Когда подходит
Локально, конвейер	Тот же компьютер, CPU, `--engine cpu` (по умолчанию)	Любой компьютер; 4–6 с на страницу
Локально, цельная модель	Тот же компьютер, GPU, `--engine gpu`	Есть видеокарта с 8 ГБ; 3,5 с на страницу
Сервер + агент	Сервер принимает пользователей; распознавание и чат выполняет подключённый компьютер-агент, при его отсутствии — CPU сервера	Публичный сайт без аренды GPU

Локально

Подготовка окружения, базовой модели и Ollama — в README. Запуск:

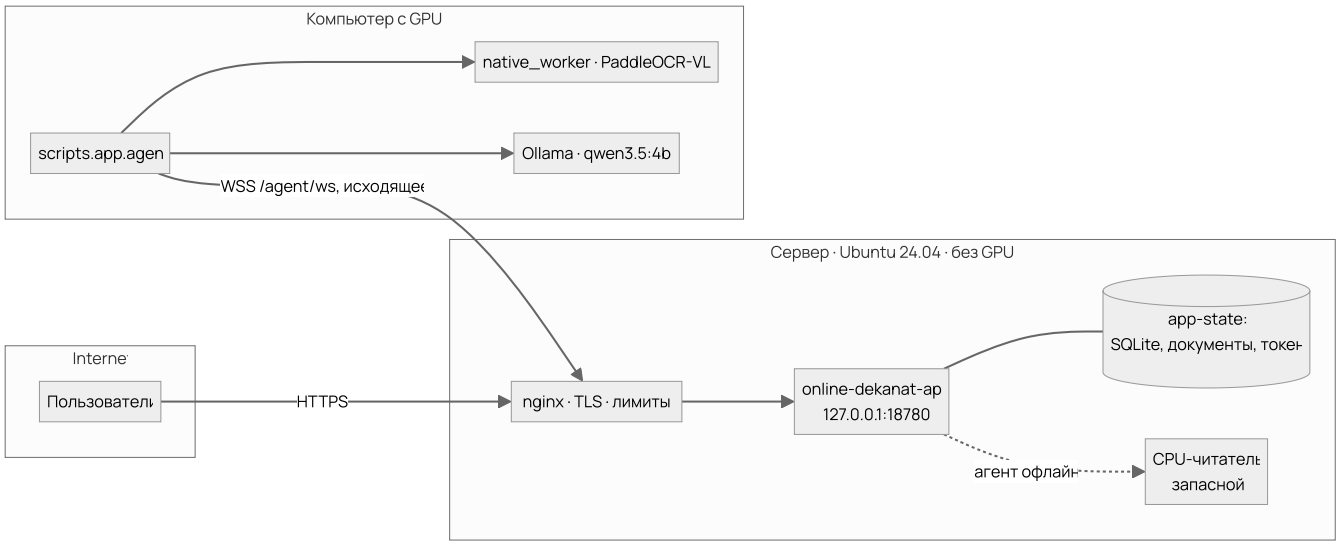
```
python -m scripts.app.server --port 8502 --engine cpu
```

Параметр	По умолчанию	
`--base`	`work/paddle-vl-base-v1`	Базовая модель
`--adapter`	`models/unified-experimental-natural-v5`	Адаптер LoRA
`--head`	`models/unified-experimental-graphic-head-fp32-v3`	Глобальная графическая голова
`--mark-head`	—	Патч-голова; без неё рамок отметок нет
`--assistant-model`	`qwen3.5:4b`	Модель Ollama; пустое значение отключает ассистента
`--work`	`work/app-v1`	SQLite и файлы документов
`--engine`	`auto`	`cpu` (= `auto`) — конвейер; `gpu` — цельная модель; `vlm-cpu` — цельная модель на CPU, только для сравнения
`--cpu-python`	`work/venv`	Интерпретатор окружения конвейера
`--host`, `--port`	`127.0.0.1`, `8502`	
`--agent-token-file`	—	Режим сервера: принимать GPU-агента на `/agent/ws`
`--secure-cookies`	выкл.	Флаг `Secure` для cookie, если сервер работает за HTTPS

Параметры `--base`, `--adapter`, `--head`, `--mark-head` относятся к цельной модели. Переменная `DEKANAT_THREADS` задаёт число потоков CPU, по умолчанию 6; остальные переменные конвейера — в pipeline.md.

На Windows то же делают `launch-app.ps1` и `launch-agent.ps1`. Каталог `work` можно переопределить переменной `DEKANAT_WORK`.

Сервер + GPU-агент



Как устроен протокол агента: architecture.md.

Установка на сервер

Требования: Ubuntu 22.04+ с Python 3.12, около 6 ГБ на диске (окружение и модель), 8+ ГБ RAM для запасного CPU-режима, nginx и сертификат Let's Encrypt.

1. Собрать архив релиза на любой машине:

   ```
   python deploy/app-v2/build_release.py app-v2.tar.gz
   ```

   В архив входят код, интерфейс и небольшие собственные модели. Базовой модели, данных и документов в нём нет.

2. Скопировать архив на сервер и выполнить от root:

   ```
   tar -xzf app-v2.tar.gz -O deploy/app-v2/install.sh > install.sh
   bash install.sh app-v2.tar.gz
   ```

Скрипт выполняет этапы по порядку. Любой этап можно запустить отдельно вторым аргументом:

Этап	Что делает
`release`	Распаковывает в `/opt/online-dekanat/releases/app-v2`. Прежний релиз переименовывается в `app-v2.prev-<время>`. Создаёт пользователя `online-dekanat`, каталог `app-state` и случайный токен агента
`venv`	`/opt/online-dekanat/venv-app`: CPU-torch и `deploy/app-v2/requirements-server.txt`
`model`	Скачивает PaddleOCR-VL 1.6 по ревизии и сверяет SHA-256 каждого файла
`service`	Устанавливает systemd unit `online-dekanat-app`, запускает его и ждёт `/api/health`
`nginx`	Делает резервную копию текущих конфигураций, ставит новый vhost и лимиты. Проверяет `nginx -t`; при ошибке возвращает прежние файлы
`switch`	Останавливает прежнюю службу web-v1. Её файлы и unit остаются для отката

Юнит systemd (`online-dekanat-app.service`):

- работает от непривилегированного пользователя;
- `ProtectSystem=strict`, запись разрешена только в `app-state`;
- `MemoryMax=9G`, `CPUQuota=450%`, 4 потока модели;
- `HF_HUB_OFFLINE=1`: на сервере модель не скачивается заново.

nginx (`nginx-site.conf`):

- TLS 1.2 и 1.3, HSTS, CSP только `self`;
- `/agent/ws` — проксирование WebSocket с таймаутом 1 час;
- `/api/` — лимит 10 запросов в секунду, потоковые ответы чата без буферизации;
- `/api/auth/` — 10 запросов в минуту;
- тело запроса до 205 МБ.

GPU-агент

На компьютере-агенте: окружение и Ollama, как для локального запуска. Для цельной модели нужны видеокарта и базовая модель. Затем скопировать токен с сервера:

```
scp root@<сервер>:/opt/online-dekanat/app-state/agent-token work/agent-v1/agent-token
```

И запустить агента (по умолчанию конвейер, `--engine gpu` — цельная модель):

```
python -m scripts.app.agent --server wss://<домен>/agent/ws --token-file work/agent-v1/agent-token
```

На Windows: `./launch-agent.ps1`.

Агент сначала загружает модель, а затем подключается. Связь он держит сам: после обрыва переподключается с задержкой от 2 до 60 с. Журнал выглядит так:

```
12:27:14 connected to wss://dekanat.karpovprod.ru/agent/ws
12:27:28 ocr result 5.5s
12:27:48 spotting result 9.8s
12:27:52 chat result 12.8s
```

Проверить режим сервера: `curl https://<домен>/api/health` возвращает `"reader": "gpu-agent"`.

Если агент не подключён:

- если связь пропала меньше минуты назад или служба только что перезапущена, задача до 20 секунд ждёт переподключения агента;
- иначе документ распознаёт конвейер на CPU сервера, около 14 секунд на страницу. Он загружается при старте службы, поэтому холодного старта нет;
- чат работает по правилам;
- как только агент подключится снова, задачи опять пойдут на него.

На одном компьютере агент и локальное приложение используют одну видеокарту. Одновременные загрузки в оба места замедлят каждое, но ошибок не вызовут.

Обновление

```
python deploy/app-v2/build_release.py app-v2.tar.gz
scp app-v2.tar.gz root@<сервер>:/tmp/
ssh root@<сервер> 'bash /root/install-app-v2.sh /tmp/app-v2.tar.gz release && bash
/root/install-app-v2.sh /tmp/app-v2.tar.gz service'
```

Данные в `app-state` при обновлении не трогаются.

Откат на прежний релиз web-v1

1. Вернуть конфигурации nginx из `/opt/online-dekanat/nginx-backup/`:
   - `site-*.conf` → `/etc/nginx/sites-available/online-dekanat`;
   - `limits-*.conf` → `/etc/nginx/conf.d/online-dekanat-limits.conf`.
2. Выполнить `nginx -t && systemctl reload nginx`.
3. Выполнить `systemctl enable --now online-dekanat`.

Диагностика

Симптом	Где смотреть
Сайт не открывается	`systemctl status online-dekanat-app`, `journalctl -u online-dekanat-app -n 50`
`reader: server-cpu`, хотя агент запущен	Журнал агента: неверный токен даёт закрытие с кодом 4401; проверьте адрес `wss://…/agent/ws`
Распознавание падает	`work/app-v1/reader.log` (локально) или `app-state/reader.log` (сервер)

Симптом	Где смотреть
Чат в «режиме правил»	Ollama: `ollama list`; модель `qwen3.5:4b` должна быть скачана
Нехватка видеопамяти	Читатель сам повторяет страницу на CPU. Закройте другие GPU-задачи

Безопасность и данные

Обработка данных

- **Локальная обработка.** Распознавание, извлечение полей и ответы ассистента выполняются собственными моделями на сервере или на подключённом агенте. Внешние API OCR и языковых моделей не используются.
- **Кадры камеры** живого режима обрабатываются в памяти и удаляются сразу после ответа.
- **Документы** хранятся в каталоге данных сервера (`documents/<slug>/`) вместе с версиями. Владелец удаляет свой черновик, сотрудник — закрытую заявку.
- **Ссылки** на заявки содержат 21 случайный символ. Знание ссылки не даёт доступа: каждая операция проверяет владельца или роль.

Роли и права

Действие	Студент	Сотрудник	Суперадмин
Загружать, проверять, отправлять свои документы	✓	✓	✓
Видеть чужие заявки (кроме черновиков)	—	✓	✓
Одобрять, возвращать, закрывать	—	✓	✓
Ставить свою подпись в редакторе	✓	✓	✓
Ставить печать деканата	—	✓	✓
Приглашения, роли, конструктор типов	—	—	✓

- Сотрудник регистрируется только по одноразовой ссылке-приглашению. Ссылка действует 7 дней и может быть отозвана. В базе хранится хэш токена.
- Суперадмин не может снять роль администратора с самого себя.

Целостность документов

- Редактор ставит только подпись текущего пользователя из его профиля и печать деканата только для сотрудников.
- Каждая правка в редакторе и каждая замена файла создают новую версию. Прежняя версия сохраняется в `history/v<n>/` и доступна деканату.
- В журнале заявки фиксируются автор и время каждого действия, а для правок в редакторе — их содержание («подпись; текст …»).
- Сервис проверяет наличие подписи и печати, но не их подлинность. Решение о принятии документа принимает сотрудник деканата.

Аутентификация

Механизм	Устройство
Пароли	PBKDF2-HMAC-SHA256 с солью
Сессии	Cookie `HttpOnly`, `SameSite=Lax`, флаг `Secure` за HTTPS; при смене пароля остальные сессии завершаются
API-ключи	`dk_live_…`; ключ показывается один раз, в базе хранится хэш
Изменяющие запросы	Проверка заголовка `Origin` для запросов из браузера
Агент	Токен в `Authorization: Bearer`, сравнение за постоянное время
Лимиты	nginx: запросы к `/api/auth/` и загрузкам ограничены по частоте

Ассистент

Ассистент отвечает только по распознанному документу и Положению. Ответ, в котором есть дата, номер или ФИО, отсутствующие в документе, отклоняется проверкой привязки. Если сведений нет, ассистент говорит об этом и направляет в деканат. Попытки заставить ассистента «принять документ» отклоняются: решение принимает сотрудник (п. 1.3 Положения).

Разработка

Окружения

В проекте три независимых Python-окружения. Их нельзя устанавливать одно поверх другого: версии transformers в них разные.

Файл	Для чего
`requirements-app.txt`	Веб-приложение, агент, цельная модель, Excel и PDF, тесты. Transformers 5.5.4
`scripts/unified/requirements-paddle.txt`	Обучение и оценка PaddleOCR-VL (то же ядро плюс `lm-format-enforcer`)
`scripts/unified/requirements.txt`	Исследование Qwen2-VL / Qwen3-VL: Transformers 4.57, bitsandbytes
`requirements.txt`, `requirements.lock.txt`	Конвейер: EasyOCR, Ultralytics, scikit-learn

Torch ставится первым, из официального индекса PyTorch под нужную версию CUDA или CPU.

Тесты

```
python -m pytest -q          # 568 тестов
python -m pytest -q tests/test_app.py tests/test_assistant.py
```

Модели в тестах заменены заглушками (`FakeReader`, подмена `ollama_chat`), поэтому GPU и веса не нужны. Покрыто:

- приложение: регистрация и роли, приглашения, сессии, загрузка, очередь, проверка `Origin`, путь заявки (отправка, рассмотрение, возврат, замена, одобрение), предупреждение и сверка ФИО, подписи и печати профиля, конструктор типов, авто-подтверждение, API-ключи;
- связь с агентом, ожидание переподключения и резервный CPU-движок;
- ассистент: контекст, вычисленные факты, проверка привязки, нормализация вида ответа, поток;
- граф-декодер, привязка полей, layout v1 и v2, проза;
- конвейер: предобработка, извлечение, нормализация, запасные правила, оценка читаемости.

Карта кода

`scripts/app/` — приложение

Файл	
`server.py`	Starlette: маршруты, авторизация, загрузка, чат, раздача интерфейса, CLI
`store.py`	SQLite: пользователи, сессии, ключи, документы
`processing.py`	Очередь с приоритетами и процесс-читатель
`pipeline.py`	Карточка документа из ответа модели; ссылки `/check/<id>`
`remote.py`	Брокер GPU-агента, удалённый читатель и чат
`agent.py`	GPU-агент
`calibrate_review.py`	Подбор порогов «Проверьте» (только оценка)

`scripts/unified/` — модель PaddleOCR-VL и исследование единой модели

Группа	Файлы
Чтение	`native_worker.py` (процесс-читатель), `fast_decode.py` (граф-декодер), `run_ocr_eval.py`, `run_graphic_strict.py`
Привязка	`text_binding.py`, `literal_layout.py`, `prose_binding.py`, `field_regions.py`, `contract.py`
Головы	`graphic_head.py`, `train_graphic_head.py`, `train_mark_patches.py`, `select_mark_rules.py`, `eval_mark_patches.py`
Генераторы данных	`prepare_v2.py`, `prepare_literal_values.py`, `prepare_natural_pages.py`, `prepare_mark_pages.py`, `prepare_real_marks.py`, `prepare_camera.py`, `fetch_natural_sources.py`
Обучение LoRA	`train_literal_ocr.py`, `train_literal_fullpage.py`, `train_literal_fp32.py`, `train_literal_values_fp32.py`, `train_natural_v5.py`
Оценка	`grade_*.py`, `compare_kit_fields.py`, `qualify_fast_decode.py`, `literal_grade.py`
Qwen2-VL / Qwen3-VL	`model.py`, `run.py`, `prepare.py`, `loss.py`, `download.py`
Прежние страницы	`local_native_web.py`, `local_native_page.py`

`scripts/assistant/` — ассистент

`core.py` (промпт, контекст, схема, проверка привязки), `http_chat.py` (поток), `eval_set.py`, `benchmark.py`, `regrade.py`, `eval_text.py`.

`scripts/participantkit/` — контракт кейса

Файл	
`format_v1.py`	Формат результата, проверка комплектности, ответы по правилам
`service.py`, `http_api.py`, `run_contract.py`	Сервис кейса, HTTP-обёртка, прогон набора
`labeled_fields.py`, `crop_recovery.py`, `value_quote_recovery.py`, `mark_guard.py`	Адаптеры извлечения конвейера
`upload_limits.py`	Проверка файла до загрузки модели: формат, мегапиксели, страницы

`dekanat/` — библиотека конвейера

`ocr.py` (EasyOCR и каскад дообученных распознавателей), `preprocessing.py` (перспектива, наклон, свет), `orientation.py`, `extraction.py`, `narrative.py`, `slots.py`, `field_validation.py`, `pdf_text*.py` (текстовый слой PDF), `visual.py` (YOLO и MobileNet), `schema.py`, `reasoning.py`, `synthetic.py`.

`scripts/*.py` — обучение и оценка моделей конвейера

`train_ocr.py`, `train_quote_ocr.py`, `train_detection.py`, `train_visual.py`, `train_fields.py`, `generate_data.py`, `evaluate*.py`, `check_document.py` (CLI одного документа), `serve_api.py` (HTTP web-v1), `build_web.py`.

Соглашения

- **Эталоны не попадают в инференс.** Скрипты чтения не открывают файлы ответов; оценка выполняется отдельными `grade_*`. Об этом говорят docstring модулей.
- **Результаты неизменяемы.** Каждый запуск пишет в новую папку и отказывается перезаписывать существующие выходы (`Fresh output required`).
- **Без выдумок.** Ни один этап не дописывает значение, которого нет в прочитанном тексте. Сомнительное помечается, отсутствующее остаётся «не найдено».
- **Внешний код не выполняется.** Модели загружаются с `trust_remote_code=False` по зафиксированным ревизиям с проверкой SHA-256.
- **Попарная проверка.** Новая версия модели или правила включается только после сравнения с текущей на тех же входах: без потерь ранее верных значений. Методика — в evaluation.md.

Как добавить тип документа

1. **Правила.** В `configs/participant_v1.json` добавить тип в `requirements`: группы обязательных ключей с номерами пунктов. Текст пунктов добавить в `rules`.
2. **Привязка.** В `scripts/participantkit/format_v1.py` добавить подписи новых полей в `LABELS`. В `scripts/unified/text_binding.py` добавить заголовок документа в `TITLES`.

3. **Интерфейс.** Добавить названия типа и полей в `web-app/assets/js/ui.js` (`TYPES`, `LABELS`), в `scripts/assistant/core.py` (`TYPE_NAMES`, `FIELD_NAMES`) и в `TYPES` сервера.
4. **Проверка.** Сгенерировать несколько страниц нового типа и проверить привязку. Если модель путает характерные значения, дообучить адаптер на новых страницах (см. training.md).

Распознавание при этом не меняется: модель читает весь текст страницы независимо от типа.